Project/Thesis No.: CSER-25-32

**CSE 4000: Thesis/Project**

# DIFFERENTIALLY PRIVATE FEDERATED LEARNING WITH ADAPTIVE CLIPPING AND NOISE SCHEDULING UNDER NON-IID DATA

By


**Tasfia Tasnim**

Roll: 1907102

**MST. Rubaiya Raktin Raha**

Roll: 1907111

**Department of Computer Science and Engineering**

Khulna University of Engineering & Technology

Khulna 9203, Bangladesh


August 2025

# Differentially Private Federated Learning with Adaptive Clipping and Noise Scheduling under Non-IID Data

By

**Tasfia Tasnim**

Roll: 1907102

&

**MST. Rubaiya Raktin Raha**

Roll: 1907111

A thesis submitted in partial fulfillment of the requirements for the degree of
"Bachelor of Science in Computer Science & Engineering"

**Supervisor:**

Signature

**Dr. M.M.A. Hashem**
Professor
Department of Computer Science and Engineering
Khulna University of Engineering & Technology

**Department of Computer Science and Engineering**

Khulna University of Engineering & Technology

Khulna 9203, Bangladesh

August 2025

# Acknowledgment

All the praise to the almighty Allah, whose blessing and mercy succeeded me to complete this thesis work fairly. I gratefully acknowledge the valuable suggestions, advice and sincere co-operation of our supervisor Dr. M. M. A. Hashem (Professor in the Department of Computer Science and Engineering) from Khulna University of Engineering & Technology. His invaluable advice, constructive feedback, and unwavering support have been key to the success of this project. We are grateful for his positive encouragement throughout. Lastly, we would like to thank our parents and family for their constant support during the difficult times of writing this research work; they have been a great motivation.

**Authors**

# Abstract

Federated Learning (FL) allows different devices to collectively train a model without sharing data. However it is not protected from privacy threats, for example, data leakage during updates. To address this risk, Differential Privacy (DP) is normally applied to protect private data.The central challenge lies in balancing privacy and utility: stronger privacy guarantees often reduce accuracy and slow convergence, particularly under realistic non-IID settings.This work investigates the integration of client-level differential privacy in FL, with the goal of minimizing the privacy–utility trade-off. We implement a federated averaging framework with adaptive clipping and compare multiple noise scheduling strategies: Flat, Low→High, and the proposed High→Low schedule. Privacy calibration is carried out using Rényi Differential Privacy accounting, with Poisson subsampling employed to mimic realistic client participation and dropout. Experiments were conducted on MNIST (IID and non-IID) and EMNIST-by-writer datasets, both of which present heterogeneous client distributions. Results show that the proposed High→Low schedule consistently achieves superior privacy–utility efficiency, maintaining accuracy close to the flat baseline while reducing privacy expenditure by up to 20–25%. In terms of accuracy per privacy budget (Acc$/\epsilon$), High→Low outperforms other schedules by up to 24.6%, demonstrating its effectiveness in harsh FL conditions.

# Contents

**PAGE**

**CHAPTER I Introduction** ..... 1

1.1 Introduction ..... 1

1.2 Background ..... 1
- 1.2.1 Challenges in Federated Learning : Attack ..... 3
- 1.2.2 Challenges of FL with DP ..... 4
- 1.2.3 Gaps in existing solutions ..... 5
- 1.2.4 How we overcome this (our contribution) ..... 5

1.3 Objectives ..... 5

1.4 Scope ..... 6
- 1.4.1 Technical / Algorithmic Tools ..... 6
- 1.4.2 Software / Programming Tools ..... 7

1.5 Unfamiliarity of the Problem ..... 7

1.6 Project Planning ..... 9
- 1.6.1 Work Plan ..... 9
- 1.6.2 Different Phases of the Project ..... 9
- 1.6.3 Work Distribution ..... 10
- 1.6.4 Applications of the Work ..... 11

1.7 Financial Analysis and Budget ..... 12

1.8 Organization of the Report ..... 13

**CHAPTER II Literature Review** ..... 14

2.1 Introduction ..... 14

2.2 Literature review ..... 14

2.3 Synthesis and research gap ..... 15

2.4 How this work addresses the gap ..... 15

2.5 Comparison table of related work ..... 16

**CHAPTER III Methodology** ..... 17

3.1 Introduction ..... 17

3.2 Problem Design and Threat Model ..... 17

3.3 Dataset and Federated Partitioning (EMNIST) ..... 17

3.4 Proposed Methodology ..... 19

3.5 Model Architecture: ..... 19
3.5.1 Federated Averaging (FedAvg) ..... 20
3.5.2 Differential Privacy Mechanism ..... 22
3.5.3 Calibration of Clients-per-Round $C_{\text{opt}}$ and Base Sigma $\sigma_0$ ..... 23
3.5.4 Noise Scheduling ..... 24
3.5.5 RDP Accounting and $\epsilon$ Reporting ..... 25
3.5.6 Evaluation protocols ..... 25

**CHAPTER IV Implementation, Results, and Discussions** ..... 30

4.1 Introduction ..... 30

4.2 Experimental Setup ..... 30

4.3 Implementation of FL and DP-FL ..... 32
4.3.1 Baseline Federated Learning (FedAvg) ..... 32
4.3.2 EMNIST Experiments ..... 34

4.4 Implementation of our proposed Method ..... 35
4.4.1 Adaptive Clipping ..... 36
4.4.2 Noise Scheduling Strategies ..... 36
4.4.3 5 Step High-to-Low noise scheduling ..... 36
4.4.4 Other Noise Scheduling Implementations ..... 39
4.4.5 Summary of Implemented Variants ..... 40

4.5 Experiment Design ..... 40

4.6 Evaluation Metrics ..... 41

4.7 Results and Analysis ..... 41
4.7.1 Noise Schedules without Validation (100 rounds) ..... 41
4.7.2 Noise Scheduling with Validation (3 runs, $T = 140$) ..... 44
4.7.3 Iso-$\epsilon$ Results (Privacy Budget $\epsilon = 3$) ..... 46

4.8 Comparison of the result ..... 48

4.9 Objective Achieved ..... 49

4.10 Summary ..... 50

**CHAPTER V Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues** ..... 51

5.1 Intellectual Property (IP) Considerations ..... 51

5.2 Ethical Considerations ..... 51

5.3 Safety Considerations ..... 52

5.4 Legal Considerations ..... 52

5.5 Impact on Societal, Health, and Cultural Issues ..... 52

5.6 Impact on the Environment and Sustainability ..... 53

**CHAPTER VI Addressing Complex Engineering Problems and Activities** ..... 54

6.1 Complex Engineering Problems ..... 54

6.2 Complex Engineering Activities ..... 55

**CHAPTER VII Conclusions** ..... 57

7.1 Challenges Faced ..... 57

7.2 Limitations of the Work ..... 58

7.3 Recommendations for Future Extensions ..... 58

# List of Tables

**PAGE**


1.1 Financial Analysis and Budget for the Project ..... 12
2.1 Summary of selected related work and relevance to this thesis. ..... 16
3.1 Comparison of Methodologies: Ours vs. Existing Approaches ..... 29
4.1 Hyperparameters on MNIST (IID and non-IID). ..... 31
4.2 Hyperparameters on EMNIST (ByWriter) and DP calibration. ..... 34
4.3 Final results of proposed High→Low noise schedule (EMNIST). ..... 38
4.4 Non-validation runs (100 rounds): schedule comparison. ..... 43
4.5 Validation-enabled runs: comparison across schedules. ..... 45
4.6 Final test accuracy and efficiency metrics. ..... 45
4.7 Iso-$\epsilon$ results with $\epsilon \approx 3$: comparison of schedules. ..... 46
4.8 Comparison of our method with existing client-level DP-FL approaches. ..... 48
6.1 Complex Engineering Problems in this Thesis. ..... 55
6.2 Complex Engineering Activities Undertaken. ..... 56

# List of Figures

**PAGE**


1.1 Gantt Chart of Project Planning ..... 10
3.1 Data distribution across the clients ..... 18
3.2 Clients and their variability in handwriting styles ..... 18
3.3 Label counts across clients ..... 19
3.4 Model Architecture ..... 21
3.5 Flowchart of Proposed Methodology ..... 27
3.6 Illustration of our Proposed Methodology ..... 28
4.1 Test accuracy of FedAvg (no DP) vs DP-FedAvg ($\sigma = 0.7$) on MNIST IID dataset. ..... 33
4.2 Test accuracy of FedAvg (no DP) vs DP-FedAvg ($\sigma = 0.9$) on MNIST non-IID dataset. ..... 33
4.3 Final metrics (Test accuracy and loss) on EMNIST (ByWriter) with FedAvg and DP-FedAvg. ..... 35
4.4 Sigma schedule over the round of the proposed 5-step High→Low schedule. ..... 37
4.5 Validation accuracy over rounds using the proposed 5-step High→Low schedule on EMNIST (ByWriter). ..... 37
4.6 Validation loss over rounds using the proposed 5-step High→Low schedule on EMNIST (ByWriter). ..... 38
4.7 Noise scheduling patterns across rounds for different strategies.. ..... 39
4.8 Privacy–utility Pareto curve annotated by round. Early rounds consume less privacy but yield rapid gains in accuracy. ..... 42
4.9 Non-validation runs (100 rounds): Comparison of schedules across different metrics. ..... 43
4.10 Comparison of 3 schedules (Flat, High→Low, Low→High) with validation. ..... 44
4.11 Iso-$\epsilon$ comparison across schedules. ..... 46
4.12 3D View: Round–$\epsilon$–Accuracy. ..... 47

# CHAPTER I

# Introduction

## 1.1 Introduction

The rapid growth of artificial intelligence and machine learning applications has led to an unprecedented demand for large datasets, which often contain sensitive personal information. This necessitates robust privacy-preserving mechanisms to ensure that the utility of these datasets can be harnessed without compromising individual privacy [1]. Federated Learning has emerged as a promising distributed machine learning paradigm that enables collaborative model training across multiple decentralized clients without direct data sharing, thereby offering an inherent level of privacy protection [2]. However, while FL prevents the direct exchange of raw data, studies have shown that privacy risks persist through the analysis of shared model updates or gradients, necessitating further protection against potential privacy violations [3] [4]. To address these vulnerabilities, Differential Privacy has been widely adopted in FL frameworks as a rigorous mathematical standard to quantify and limit the privacy leakage, particularly against inference attacks that aim to reconstruct sensitive training data from model parameters [5]. This integration of Differential Privacy into Federated Learning, often termed DP-FL, introduces quantifiable noise into the model updates or gradients before aggregation, thereby obfuscating individual contributions and safeguarding against re-identification or attribute inference [6]. This noise injection, while crucial for privacy, introduces a fundamental trade-off with model utility, requiring careful calibration to balance privacy guarantees with acceptable model performance [7].Differential privacy works by tracking overall privacy loss with an accountant (e.g., Rényi DP)[8][9].The overriding concern is balance. More privacy (more noise and clipping) is protective of clients but typically decreases precision and shrinks convergence rates. This balance is harder when realistic non-IID data is considered, where clients' label distributions are vastly different, and data and writing styles vary (i.e., EMNIST by writer). In these situations, some of the clients produce large, fluctuating updates and others are small and stable; a hard single clip bound and an unchanging noise level usually do poorly across the board[10].

## 1.2 Background

**Federated Learning (FL):** FL trains a single global model across many clients without collecting raw data at a central server . In each round $t$, the server broadcasts the current model $w^{(t)}$; selected clients perform local training on their private data and return updates; the server aggregates them to form $w^{(t+1)}$. FL reduces central data exposure but does not eliminate leak-

age from model updates.

**Differential Privacy (DP):** DP limits how much the output of a randomized mechanism $M$ can change when a single "neighboring" unit in the dataset is modified. *Pure* $\varepsilon$-DP: for all measurable sets $S$,

$$\Pr[M(D) \in S] \;\le\; e^{\varepsilon}\, \Pr[M(D') \in S], \tag{1.1}$$

where $D$ and $D'$ differ in one user (client-level DP) or one record (example-level DP). *Approximate* $(\varepsilon, \delta)$-DP:

$$\Pr[M(D) \in S] \;\le\; e^{\varepsilon}\, \Pr[M(D') \in S] \;+\; \delta. \tag{1.2}$$

We focus on *central* (server-side) DP for FL (clip and add noise after aggregation). *Local* DP (each client adds noise before sending) offers stronger trust assumptions but usually lower utility. Our target is *client-level* DP (protect a whole user's participation).DP works in two ways. Example level DP protects individual records within a client, ensuring that the presence or absence of a single data point does not significantly affect the output.whether client level DP protects entire clients, guaranteeing that participation (or absence) of one client with all of their local data does not substantially influence the global model. There are some parameters of Differential Privacy, They are:

- $\epsilon$ (epsilon): The privacy budget. Smaller $\epsilon$ means stronger privacy, but typically lower accuracy.
- $\delta$ (delta): The failure probability. Represents the chance that the privacy guarantee does not strictly hold, usually set very small (e.g., $10^{-5}$).
- $\sigma$ (sigma): The noise multiplier. Scales the Gaussian noise added to updates; higher $\sigma$ improves privacy at the expense of model utility.

**Sensitivity and noise mechanisms:** Let $f(D)$ be the (clipped) aggregated update. The $\ell_1$ sensitivity $\Delta_1 = \max_{D \sim D'} \|f(D) - f(D')\|_1$ and the $\ell_2$ sensitivity $\Delta_2 = \max_{D \sim D'} \|f(D) - f(D')\|_2$ bound how much one neighboring change can affect the output. *Laplace mechanism* (pure DP) releases $f(D) + \mathrm{Lap}(0, b)$ with $b = \Delta_1/\varepsilon$. *Gaussian mechanism* ($(\varepsilon, \delta)$-DP) releases $f(D) + \mathcal{N}(0, \sigma^2 I)$ with the classical calibration

$$\sigma \;\ge\; \frac{\Delta_2}{\varepsilon}\sqrt{2\ln\!\big(1.25/\delta\big)}. \tag{1.3}$$

In practice we use Gaussian noise; modern RDP-based calibrations are tighter.

**Clipping and adaptive clipping:** Each client update $u$ is clipped to limit influence:

$$\tilde{u} \;=\; u \cdot \min\!\left(1,\; \frac{C}{\|u\|_2}\right), \tag{1.4}$$

so every contribution has norm $\leq C$. A single fixed $C$ often fails under non-IID data: it over-clips informative large updates or under-clips outliers. *Adaptive clipping* adjusts the bound using recent update norms. Common variants include percentile-based (set $C_t$ to a running percentile, e.g., 80th), EMA-based (exponential moving average of typical norms), and per-layer or per-group scaling. Adaptive clipping stabilizes training on non-IID data while keeping DP calibration tied to a realistic bound.

**Noise scheduling (adaptive noise across rounds):** Instead of a constant noise multiplier, vary $\sigma$ by round $t$: flat ($\sigma_t = \sigma_0$), high→low (more noise early, less later), or low→high (the reverse). Some works allocate noise per-parameter (e.g., less noise to important parameters). Our focus is round-wise scheduling to spend privacy where it hurts utility least.

**Rényi Differential Privacy (RDP) and accounting:** RDP measures privacy at order $\alpha > 1$ with a value $\varepsilon_\alpha$ that *adds under composition* across rounds. After summing $\varepsilon_\alpha$ over rounds, convert to $(\varepsilon, \delta)$ via

$$\varepsilon(\delta) \;=\; \min_{\alpha>1} \left\{ \varepsilon_\alpha \;+\; \frac{\ln(1/\delta)}{\alpha - 1} \right\}. \tag{1.5}$$

Sampling a random fraction of clients each round provides privacy amplification. We report running $\varepsilon$ at a fixed $\delta$.

**FedAvg (algorithm and equation):** In FedAvg, clients take several local steps before sending updates. With client $k$ holding $n_k$ records and $S_t$ the selected clients, the server aggregates either by model averaging

$$w^{(t+1)} \;=\; \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j}\, w_k^{(t,\mathrm{local})} \tag{1.6}$$

or update averaging

$$w^{(t+1)} \;=\; w^{(t)} \;+\; \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j}\, \Delta w_k^{(t)}. \tag{1.7}$$

With central DP, we aggregate *clipped* updates and add Gaussian noise to the aggregate before updating the model. A common DP update is:

$$\hat{g}(t) \;=\; \frac{1}{|S_t|} \sum_{k \in S_t} \tilde{u}_k \;+\; \mathcal{N}\big(0,\, \sigma_t^2\, C_t^2\, I\big), \qquad w^{(t+1)} \;=\; w^{(t)} \;+\; \eta\, \hat{g}(t). \tag{1.8}$$

### 1.2.1 Challenges in Federated Learning : Attack

- **Gradient/model inversion.** Reconstruct features of client data from shared gradients or model deltas, especially early in training or with small batches.
- **Membership inference.** Test whether a specific user's data was part of training using the global model or round snapshots.

- **Property inference.** Infer sensitive attributes correlated with the data (e.g., writer demographics, device traits) from updates or model behavior.
- **Data/label poisoning.** Inject crafted examples or flip labels to degrade accuracy or steer predictions.
- **Backdoor/model replacement.** Implant a hidden behavior (trigger $\rightarrow$ target class) while keeping global accuracy high.
- **Byzantine updates.** Arbitrary or adversarial updates destabilize aggregation and derail convergence.
- **Sybil and free-rider attacks.** Many fake clients to sway aggregation, or clients that send zero/random updates to exploit incentives or waste privacy budget.

### 1.2.2 Challenges of FL with DP

- **Utility–privacy coupling under non-IID.** Noise and clipping interact with skewed client distributions: informative large updates are clipped; added noise disproportionately harms rare classes.
- **Hyperparameter entanglement.** The trio $(C_t,\ \sigma_t,\ q_t)$ jointly determines privacy and optimization: small $C_t$ reduces sensitivity but adds bias; large $\sigma_t$ protects privacy but increases variance; low $q_t$ helps privacy via amplification but slows learning.
- **Dynamic participation and accounting drift.** Varying clients per round yield fluctuating $q_t = K_t/|D|$, changing per-round privacy and making accurate $\varepsilon$ tracking essential.
- **Fairness and representational harm.** Clients with few samples or minority classes can be hit harder by noise/clipping, yielding uneven accuracy even at the same global $\varepsilon$.
- **Layer/parameter sensitivity mismatch.** One global clip and scalar noise do not match heterogeneous sensitivity across layers; critical layers can be drowned or biased.
- **Robustness vs. DP tension.** Strong DP reduces the signal-to-noise ratio that robust aggregation relies on, hampering detection of anomalous/malicious updates.
- **Implementation risk.** Silent issues (wrong clip units, pre/post-noise scaling, accountant misuse) can invalidate claimed $(\varepsilon, \delta)$; per-round logs of $C_t,\ \sigma_t,\ q_t$ and accountant state are often missing.

### 1.2.3 Gaps in existing solutions

- **Uncoordinated levers.** Most works keep fixed clipping with constant noise, or study either adaptive clipping or noise scheduling—not both, jointly tuned across rounds.
- **Round-level transparency.** Few report per-round $C_t,\ \sigma_t,\ q_t$ or show Accuracy-vs-$\varepsilon$ trajectories; single end-point reporting hides trade-off dynamics.
- **Benchmark realism.** Heavy reliance on (near-)IID MNIST/Fashion-MNIST; fewer results on *EMNIST by writer* or real device splits.
- **Accounting comparability.** Different accountants/$\delta$ make $\varepsilon$ numbers not directly comparable; iso-$\varepsilon$ evaluations at a fixed $\delta$ are uncommon.
- **Granularity mismatch.** Parameter-wise noise can help but often lacks a round-wise plan; round schedules ignore layer sensitivity—neither fully bridges both.
- **Attack-aware evaluation.** Privacy/utility rarely evaluated alongside robustness (inversion, membership, backdoors) under non-IID.
- **Reproducibility.** Missing seeds, privacy logs, and accountant/scheduler code hinder validation of $(\varepsilon, \delta)$ claims and replication.

### 1.2.4 How we overcome this (our contribution)

- **Coordinate the two main levers.** Couple *adaptive clipping* (match real update scales each round) with *round-wise Gaussian noise scheduling* (e.g., high→low) to spend privacy where it costs less in utility.
- **RDP-based calibration and auditability.** Use Rényi DP composition with subsampling to target a fixed $\delta$ and report the running $\varepsilon$; log $C_t,\ \sigma_t,\ q_t$ each round.
- **Realistic non-IID evaluation.** Evaluate on *EMNIST by writer*; compare flat vs. scheduled noise under iso-$\varepsilon$; include an ablation of adaptive vs. fixed clipping.
- **Clear, comparable metrics.** Provide Accuracy vs. Rounds, $\varepsilon$ vs. Rounds, and Accuracy at matched $\varepsilon$, plus per-round logs, so results are reproducible and directly comparable.

## 1.3 Objectives

Our main objective is to balance privacy and accuracy in cross-device federated learning on realistic non-IID data by adding adaptive clipping with round-wise Gaussian noise scheduling, evaluated under server-level DP.

**Specific goals:**

- To design and implement a DP-FL pipeline with adaptive clipping, scheduled noise (flat or high→low or low→high), and Rényi-DP accounting.
- To compare the three noise schedules under the same privacy budget $(\varepsilon, \delta)$, measuring effects on accuracy and convergence.
- To quantify the privacy–utility trade-off on EMNIST (by writer): report accuracy, loss, final $\varepsilon$ at a fixed $\delta$, and plot accuracy-vs-$\varepsilon$ curves.
- To compare adaptive clipping with fixed clipping at matched $(\varepsilon, \delta)$.
- To provide practical settings (clip range $C_t$, base noise $\sigma_0$, schedule multipliers, client fraction $q_t$, local epochs) that maintain strong privacy with minimal accuracy loss.

**Success criteria:**

- At a fixed $(\varepsilon, \delta)$, adaptive clipping + scheduled noise achieves higher privacy than fixed clip + constant noise.
- Stable training on non-IID EMNIST with clear per-round logs of $C_t$, $\sigma_t$, and the accountant's running $\varepsilon$.
- Reproducible results (fixed seeds, reported hyper-parameters, and plotted metrics).

### 1.4 Scope

#### 1.4.1 Technical / Algorithmic Tools

**Federated learning tools**

- **FedAvg** — cross-device FL with multiple local epochs and weighted aggregation.
- **DP-SGD / DP aggregation** — per-client *clipping* with *Gaussian noise* added at the *server* (client-level DP).
- **Adam optimizer** — local training optimizer on clients for faster, stable convergence under non-IID data.
- **Rényi DP (RDP) accounting** — per-round privacy composition (with subsampling amplification) and conversion to $(\varepsilon, \delta)$; reported as running $\varepsilon$.

**Machine learning tools**

- **Model architecture:** Flatten → Dense(200, ReLU) → Dense(200, ReLU) → Dense(10, logits).

- **Loss & metrics:** Sparse Categorical Cross-Entropy (from logits), Sparse Categorical Accuracy.
- **Data handling:** EMNIST (by writer) non-IID split; normalization; client-level train/validation separation (validation clients never spend privacy budget).
- **Client sampling:** Poisson subsampling per round (fraction $q_t$).
- **Privacy controls:** global *adaptive clipping* $C_t$ (percentile/EMA style); *noise scheduling* over rounds (flat / high→low / low→high); target $\delta$ fixed (e.g., $10^{-5}$).
- **Training control:** fixed seeds for reproducibility; round budget / early stopping as appropriate; per-round logs of $C_t$, $\sigma_t$, $q_t$, and running $\varepsilon$.

### 1.4.2 Software / Programming Tools

- **TensorFlow Federated (TFF)** — FL orchestration (client sampling, local training, server aggregation, DP aggregator).
- **TensorFlow / Keras** — model definition, local update steps, and metrics.
- **Google `dp_accounting`** — RDP composition and $(\varepsilon, \delta)$ conversion.
- **TensorFlow Datasets (TFDS)** — loading EMNIST (by writer) and MNIST (sanity checks).
- **Python (NumPy, Pandas)** — preprocessing, metrics aggregation, CSV logs.
- **Matplotlib** — plots: accuracy vs. rounds, $\varepsilon$ vs. rounds, accuracy vs. $\varepsilon$.
- **Google Colab** (GPU when available) — execution environment for experiments.
- **LaTeX** — thesis formatting and vector block diagrams (server-side DP, local-side DP, model).

## 1.5 Unfamiliarity of the Problem

This project tackles a challenge not typically found in standard coursework: balancing *client-level differential privacy* with model accuracy in cross-device federated learning, especially under realistic non-IID conditions. Our approach brings together *adaptive clipping* and *round-wise noise scheduling*, and we validate it on the EMNIST dataset split by writer. While many classes and tutorials introduce basic concepts like differential privacy $(\varepsilon, \delta)$, DPSGD, and FedAvg using simple IID datasets, they do not explore how to jointly tune adaptive clipping with per-round Gaussian noise schedules or how to set up a full evaluation protocol with iso-$\varepsilon$ comparisons and detailed per-round Rényi DP (RDP) tracking.

**Why our topic and solution are different:**

- **Synthesis, not template:** We did not follow a pre-designed lab. Our solution integrates adaptive clipping, noise scheduling across rounds, and RDP-based privacy calibration with subsampling—elements that are usually taught separately and not combined in practice.
- **Focus on realistic non-IID data:** Unlike typical course examples that use IID MNIST with fixed noise and clipping, we use EMNIST by writer, a dataset with real-world heterogeneity in labels and writing styles. This setting exposes the failure of "one-size-fits-all" privacy parameters and is rarely used in coursework.
- **Custom noise scheduling:** We implemented and tuned a five-step high-to-low noise schedule and compared it against flat and low-to-high alternatives. There is no built-in "scheduler" function in standard libraries; this was custom-designed.
- **Transparent privacy accounting:** We log the clipping bound $C_t$, noise level $\sigma_t$, and client sampling rate $q_t$ for each round, along with the running privacy loss $\varepsilon$ at a fixed $\delta$. Most course materials do not go beyond reporting a final $\varepsilon$, and rarely require transparency at the round level.
- **Evaluation at matched privacy budgets:** We run all comparisons under the same $(\varepsilon, \delta)$ constraints and include an ablation comparing adaptive and fixed clipping. This kind of iso-$\varepsilon$ evaluation is not standard in coursework.
- **Independent design choices:** All parameters (clipping values, noise scale, step size, client sampling, and evaluation metrics) were selected and tuned by us based on what worked in the non-IID setting; we did not copy an existing recipe.
- **Deeper threat analysis:** We go beyond simply adding noise and measuring accuracy. Our work considers how privacy settings interact with known federated learning attacks such as inversion, membership inference, and poisoning—areas often overlooked in classroom settings.

.

## 1.6 Project Planning

### 1.6.1 Work Plan

To ensure the effective and organized execution of the project, a clear work plan is essential. The project plan outlines various phases of the work and sets milestones to track progress effectively. These phases include planning, implementation, evaluation, reporting, and conclusion. Each phase is designed to ensure that the project moves forward smoothly and meets its objectives on time.

### 1.6.2 Different Phases of the Project

The project will be executed in several phases, each with distinct goals and deliverables:

1. **Planning Phase**:
    - This phase involves defining the project scope, reviewing relevant literature, and finalizing the methodology for implementing the federated learning system with differential privacy.
    - Preparation of necessary planning documents and setting up tools for the execution phase.
2. **Implementation Phase**:
    - Development of the adaptive clipping and noise scheduling mechanism, which is central to the project.
    - Integration of these mechanisms with federated learning algorithms and the addition of privacy-preserving techniques using DP.
    - Testing and tuning the methods to ensure proper functionality and alignment with privacy goals.
3. **Evaluation Phase**:
    - Running experiments to assess the effectiveness of adaptive clipping and round-wise noise scheduling.
    - Comparing different noise scheduling methods and analyzing privacy-utility trade-offs using the EMNIST dataset.
    - Ensuring the system meets the set privacy budget while maintaining model accuracy.
4. **Reporting Phase**:

- Documenting the methodology, results, and analysis.
- Writing the thesis report, which includes all the experimental results, comparisons, and conclusions drawn from the work.

5. **Conclusion Phase**:
    - Final review of the work, ensuring all objectives have been met.
    - Preparation for the thesis defense, which involves summarizing the project's contributions and outcomes.

Fig. 1.1 provides a detailed visual representation of the time frame for these activities. This planning tool will help ensure that all tasks are completed within the allotted time, maintaining alignment with project goals.

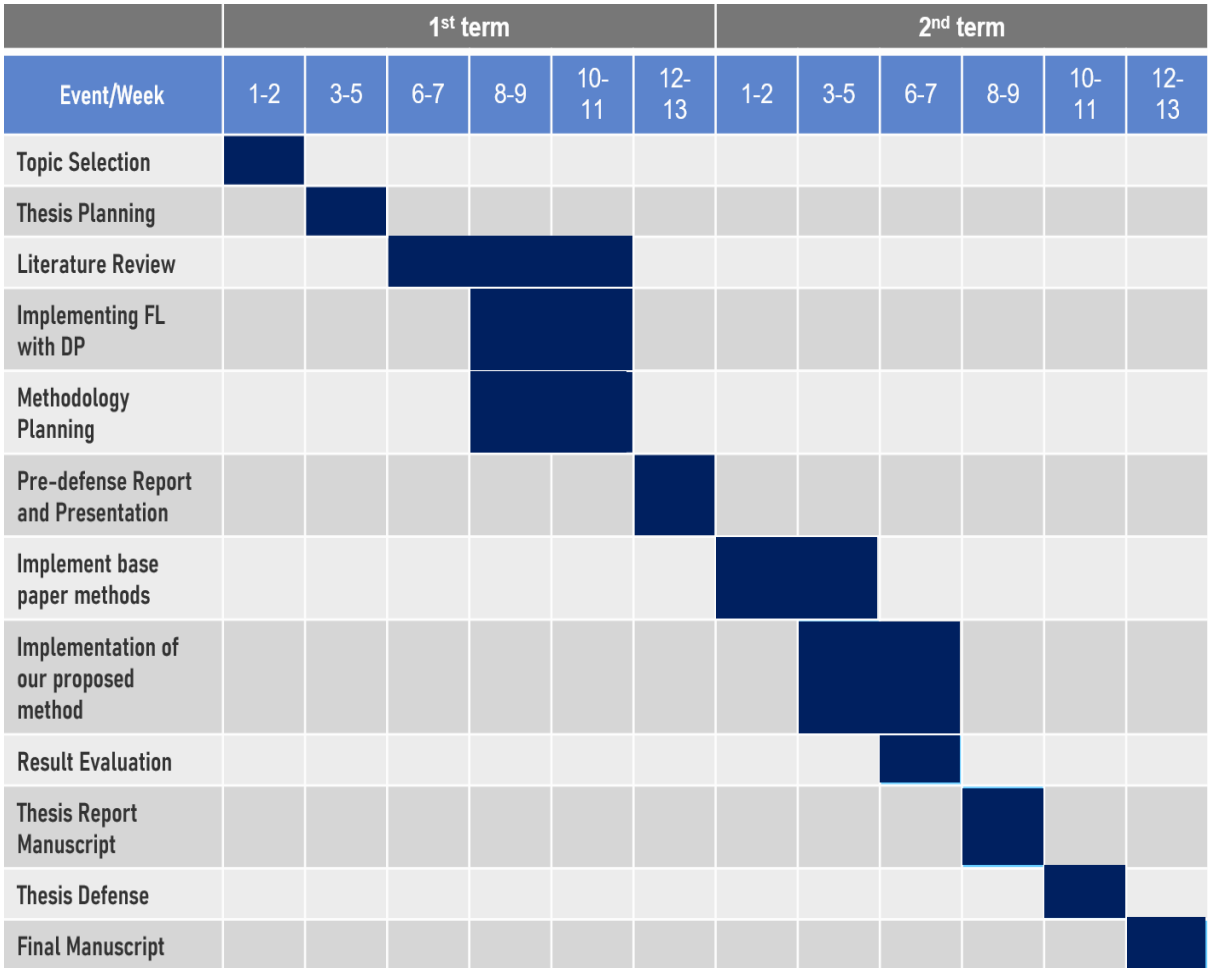



Figure 1.1: Gantt Chart of Project Planning

### 1.6.3 Work Distribution

**Tasfia Tasnim (Roll:1907102) :** Tasfia focused more on the theoretical and foundational aspects of the thesis, carrying out an extensive literature review on Federated Learning, privacy

risks, and prior work on DP accounting, which laid the groundwork for this research. She formalized the problem design and threat model, and prepared significant portions of the background and methodology chapters. On the implementation side, she conducted the MNIST experiments in both IID and non-IID settings, providing strong baselines. She also worked with the privacy accounting components, including Rényi DP, and supported the interpretation of results across different schedules.

**Rubaiya Raha (Roll:1907111)** Raha contributed to both the theoretical study and the implementation of the work. She was actively involved in the literature review on Differential Privacy mechanisms and federated optimization, particularly focusing on noise scheduling and adaptive clipping approaches. On the implementation side, she designed and developed the core methodology in TensorFlow Federated, including adaptive clipping, noise scheduling strategies (Flat, Low→High, High→Low), and the EMNIST experiments under harsh non-IID conditions. Raha also performed detailed result analysis, highlighting privacy–utility trade-offs, and contributed extensively to the experimental chapters of the thesis.

Together, both authors made equally valuable contributions—Raha driving the implementation and methodological innovations, and Tasfia ensuring strong theoretical grounding and baseline validation—resulting in a well-balanced and comprehensive thesis.

### 1.6.4 Applications of the Work

Differential Privacy in Federated Learning has a wide range of applications across various domains, thanks to its ability to provide privacy-preserving machine learning in decentralized settings. Some prominent applications include

- **Biomedicine and healthcare.** Hospitals and research labs can collaborate to train diagnosis or triage models without exposing patient data. By combining client-level differential privacy with adaptive clipping and time-scheduled noise, our approach protects sensitive or rare conditions (including underrepresented populations) while maintaining strong accuracy across highly non-IID hospital datasets.

- **Finance and fintech.** Banks and digital wallets can jointly develop models for fraud detection, credit risk scoring, or anti–money laundering while keeping customer information private. The integration of RDP tracking and iso-$\varepsilon$ reporting in our approach also supports compliance with regulatory frameworks such as GDPR.

- **Mobile and IoT personalization.** On-device tools—keyboards, voice assistants, and recommender systems—benefit from server-side differential privacy with adaptive clipping, especially when user behavior varies widely. A high→low noise schedule lowers the privacy cost during early training while preserving accuracy as the model converges.

- **Enterprise federated analytics.** Retail chains or logistics hubs across different locations can share insights through federated learning without revealing raw operational logs. Our method provides tested hyperparameter guidelines—clipping thresholds, base noise levels, and noise schedules—for stable training under skewed or imbalanced client participation.

- **Wearables and smart devices.** Devices like home cameras, smartwatches, and other IoT gadgets can contribute model updates securely. Adaptive clipping handles noisy sensor data, while round-wise noise maintains privacy without severely degrading model performance.

- **Public sector and research partnerships.** Universities, government agencies, and public hospitals working with sensitive data (education, transportation, epidemiology) can use our approach to collaborate securely. The use of iso-$\varepsilon$ comparisons ensures that results remain interpretable and consistent across multiple stakeholders.

- **Safety-critical model updates.** In industrial monitoring or clinical decision support, applying stricter noise and tighter clipping in the early rounds helps reduce leakage risks during the most vulnerable phases of training.

- **Methodological contribution.** Beyond any single domain, our work introduces reusable techniques: how to scale noise up or down over time, how to set adaptive clipping for non-IID scenarios, and how to maintain per-round logs of clipping bounds, noise levels, client participation rates, and the evolving $\varepsilon$—all to ensure transparent and auditable privacy accounting.

### 1.7 Financial Analysis and Budget

All software tools used, including TensorFlow Federated, are open-source. Google Colab's free tier has been used to perform computations, which minimizes the financial burden for implementation.

Table 1.1: Financial Analysis and Budget for the Project

| **Category** | **Item** | **Cost (Tk)** |
|---|---|---|
| Compute Resources | Google Colab (free tier) | 0 |
| Dataset Acquisition | MNIST / EMNIST (public) | 0 |
| Software | Python, TensorFlow, TFF, LaTeX | 0 |
| Documentation | Printing and binding | 800 |
| Miscellaneous | Internet and utilities | 400 |
| | Contingency (10% of paid items) | 100 |
| **Total Estimated Budget** | | **1300** |

## 1.8 Organization of the Report

Chapter I introduces the problem of privacy-preserving federated learning, outlines objectives, scope, applications, and planning. Chapter II reviews FL and DP foundations, related systems, and identifies research gaps. Chapter III presents the methodology, including dataset, model, adaptive clipping, noise scheduling, and Rényi DP accounting. Chapter IV details the implementation in TensorFlow Federated, reports experimental results, and compares noise schedules with ablations. Chapter V discusses societal, health, safety, legal, and cultural issues of deploying DP-FL. Chapter VI addresses complex engineering problems and the activities undertaken to solve them. Chapter VII concludes with key findings, limitations, and future work, followed by the References section listing all cited works.

# CHAPTER II

# Literature Review

## 2.1 Introduction

Federated learning (FL) lets many clients train a shared model without sending raw data to a server. This is useful for privacy, but model updates can still leak information. Differential privacy (DP) is a principled way to limit this leakage by adding noise to updates and keeping a formal budget $(\varepsilon, \delta)$. The challenge is balance: Too much noise hurts accuracy; too little noise weakens privacy. On realistic non-IID data, this balance is even harder because client updates vary widely. This chapter reviews work on DP in FL, adaptive clipping, and noise scheduling, and then places our thesis, balancing privacy with adaptive DP and accuracy in FL under realistic non-IID data, in that context.

## 2.2 Literature review

Federated learning (FL) first appeared as a way to train a single global model without collecting raw data in one place, improving scalability and reducing central privacy risk (surveyed in [11]). Soon after, the community turned to differential privacy (DP) to give formal protection against information leakage from model updates: client updates are clipped and calibrated noise is added, while total privacy loss over many rounds is tracked (e.g., moments or Rényi accounting). Early uses of DP were in centralized deep learning with DP-SGD; these ideas were then brought into FL as DP-FedAvg / DP-FedSGD by combining clipping and Gaussian (or Laplace) mechanisms during aggregation. Most recently, FL-ODP extended this line by systematically varying the noise level $\sigma$ and the parameter $\delta$ under a fixed $\varepsilon$ on non-IID MNIST, reporting clear accuracy–privacy trade-offs but not exploring adaptive clipping or round-wise noise schedules [12].

Fixed clipping bounds often perform poorly in non-IID FL since client update magnitudes vary widely. Adaptive clipping adjusts the bound dynamically using gradient norm statistics. Andrew et al. showed that adaptive clipping can outperform fixed clipping in DP-SGD by reducing bias from over-clipping and stabilizing training, but their work focused on centralized settings and did not combine with round-wise noise schedules [13]. Thakkar et al. deployed adaptive clipping in Google's production DP-FedAvg for Gboard next-word prediction, demonstrating large-scale feasibility, but again without structured scheduling of noise [14]. Personalized FL with DP similarly adapts to user heterogeneity by adding noise to shared updates while using moments-based accounting; it shows improved robustness under device variability, but does not coordinate noise per round [15].

Rather than using a constant noise multiplier, noise can be varied across rounds. Bu et al. proposed adaptive noise scaling, adjusting $\sigma$ over training rounds to optimize the $\varepsilon$–utility trade-off, showing that schedule-aware privacy can improve deep learning accuracy; however, this was not applied in federated setups or combined with adaptive clipping [16]. Wu et al. studied privacy budget allocation in convex collaborative ML, showing theoretically that varying noise across rounds affects convergence, but their analysis was limited to convex models and synthetic data [17]. Empirical FL work like DDPFL instead assigns differentiated noise *per parameter* using importance coefficients, improving accuracy under the same privacy budget on MNIST and Fashion-MNIST, but without per-round scheduling [18].

Many studies, including FL-ODP and DDPFL, evaluate on MNIST or Fashion-MNIST, which are easier and often IID-split. Wu's and Bu's works also lack non-IID federated tests. MDPFLA explores layering multiple DP protections across FL, but provides limited empirical results on challenging non-IID cases [19]. By contrast, EMNIST introduces strong heterogeneity from diverse writers and label imbalance. Our thesis targets this harder setting to measure the real impact of combining adaptive clipping with round-wise noise schedules.

Overall, the literature has shown the value of adaptive clipping and noise structuring separately, but a systematic study that unifies both in federated learning under realistic non-IID data remains missing.

### 2.3 Synthesis and research gap

From these works we learn: (a) DP in FL is feasible but requires careful tuning of $\varepsilon$, $\delta$, clipping, and noise; (b) adaptive clipping clearly improves utility under non-IID data compared to fixed thresholds; (c) noise can be structured either across parameters (DDPFL) or across rounds (Bu, Wu), but not often both; (d) most evaluations are on MNIST or synthetic data, not on harder EMNIST. The remaining gap is a systematic study that coordinates adaptive clipping with a round-wise noise schedule and measures the privacy–utility path on realistic non-IID EMNIST.

### 2.4 How this work addresses the gap

We use adaptive clipping to stabilize updates and a scheduled Gaussian noise plan to align noise with training dynamics. We account for total privacy loss across rounds using RDP and report utility at the same $(\varepsilon, \delta)$ budget. By focusing on EMNIST non-IID, we reflect real-world heterogeneity. This directly targets the central trade-off shown in prior work (higher noise lowers disclosure but may hurt accuracy) by *allocating* noise where it costs less in utility [12, 13, 16].

## 2.5 Comparison table of related work

Table 2.1: Summary of selected related work and relevance to this thesis.

| **Work** | **Dataset(s)** | **Method focus** | **Key idea / claim** | **Limitation (in context)** | **Relevance here** |
|---|---|---|---|---|---|
| FL-ODP [12] | MNIST (non-IID), FedAvg | Tune DP params under fixed $\varepsilon$ | Vary $\sigma$ and $\delta$; report accuracy–privacy trade-offs | MNIST only; no adaptive-clip + per-round schedule | Baseline trade-off we extend |
| DDPFL [18] | MNIST, FMNIST | Differentiated noise per parameter | Importance coefficients; less noise to important params | Not combined with adaptive clipping + round schedule | We structure noise across rounds |
| Personalized FL w/ DP [15] | Mobile sensing (IoT) | Personalization + DP; moments accountant | Noise on shared updates; robust under device variability | Personalization focus; no round-wise scheduling | We study global model + schedule |
| MDPFLA [19] | General FL summary | Multiple DP protections | Layered DP across FL steps | Higher complexity; non-IID results limited | Layered DP $\neq$ schedule + adaptive clip |
| Adaptive clipping (Andrew et al.) [13] | MNIST, CIFAR (centralized) | Adaptive clipping of updates | Privately estimated clip bound improves training stability | Not federated; no noise scheduling | We combine adaptive clip + scheduling in FL |
| Google DP-FedAvg (Thakkar et al.) [14] | Gboard text prediction | Production FL with DP | Adaptive clipping + moments accountant at scale | Production-focused; no structured schedule | We extend ideas to EMNIST with scheduling |
| GDP (Bu et al.) [16] | MNIST, CIFAR-10 (centralized) | Adaptive noise scaling | Adjust $\sigma$ over rounds for better $\varepsilon$–utility balance | Not FL; no adaptive clipping | We bring schedules into FL + adaptive clipping |
| Collaboration + DP (Wu et al.) [17] | Synthetic convex ML | Privacy budget allocation | Varying noise across rounds affects convergence | Convex ML only; no adaptive clipping; not FL | We show deep-learning FL evidence |
| **This thesis** | **EMNIST (non-IID)** | **Adaptive clipping + per-round noise schedule** | **Coordinate clipping and noise under fixed** $(\varepsilon, \delta)$ | — | **Directly addresses open gap** |

# CHAPTER III

# Methodology

## 3.1 Introduction

The primary objective of this research is to minimize the privacy-utility trade-off in Differential Privacy-enabled Federated Learning systems. Our approach leverages Differential Privacy to ensure client level privacy in federated learning set up, while minimizing the privacy-utility trade-off. We first discuss the data partitioning and client splitting process, followed by our proposed methodology with an explanation of the learning algorithm (FedAvg) with Differential privacy mechanisms. Then, we dive into noise scheduling strategies and their impact on the model accuracy, comparing flat, low-to-high and high-to-low schedules. Finally we discuss the calibration of privacy parameters, along with validation based early stopping to prevent overfitting. The section concludes with evaluation protocols and the metrics used to measure privacy-accuracy trade off.

## 3.2 Problem Design and Threat Model

We train a cross-device federated classifier under client-level differential privacy. The server coordinates many user devices (clients) without collecting raw data. The privacy objective is to guarantee $(\epsilon, \delta)$-differential privacy at the client level, with the reporting privacy set at $\delta = 10^{-5}$. When indicated, we enforce an $\epsilon$-cap (e.g., $\epsilon_{\text{max}} = 3.0$) to ensure that the privacy of each endpoint is matched (i.e., iso-$\epsilon$).

## 3.3 Dataset and Federated Partitioning (EMNIST)

We train on federated EMNIST built by writer identity, a real-world cross-device scenario with strong non-IID features. Clients suffer from extreme label skewness (most contain just one or two classes), heavy-tailed counts, and variability in handwriting styles. Figure 3.1 represents the data distribution across the client; Figure 3.2 reports three clients with their own handwritten digits; Figure 3.3 measures distributional difference of labels across clients. This scenario is deliberately challenging to DP-FL, straining both optimization and privacy accounting.

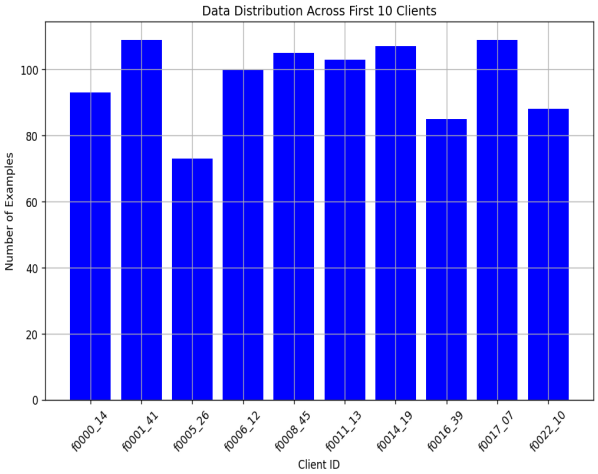



Figure 3.1: Data distribution across the clients

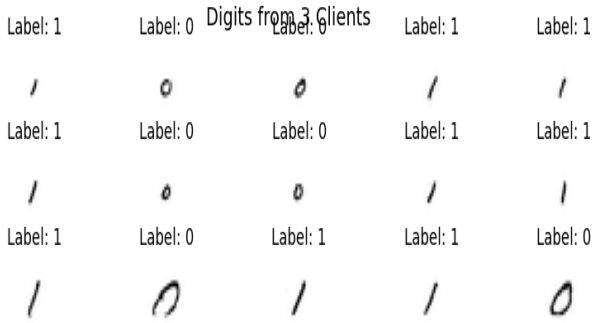



Figure 3.2: Clients and their variability in handwriting styles

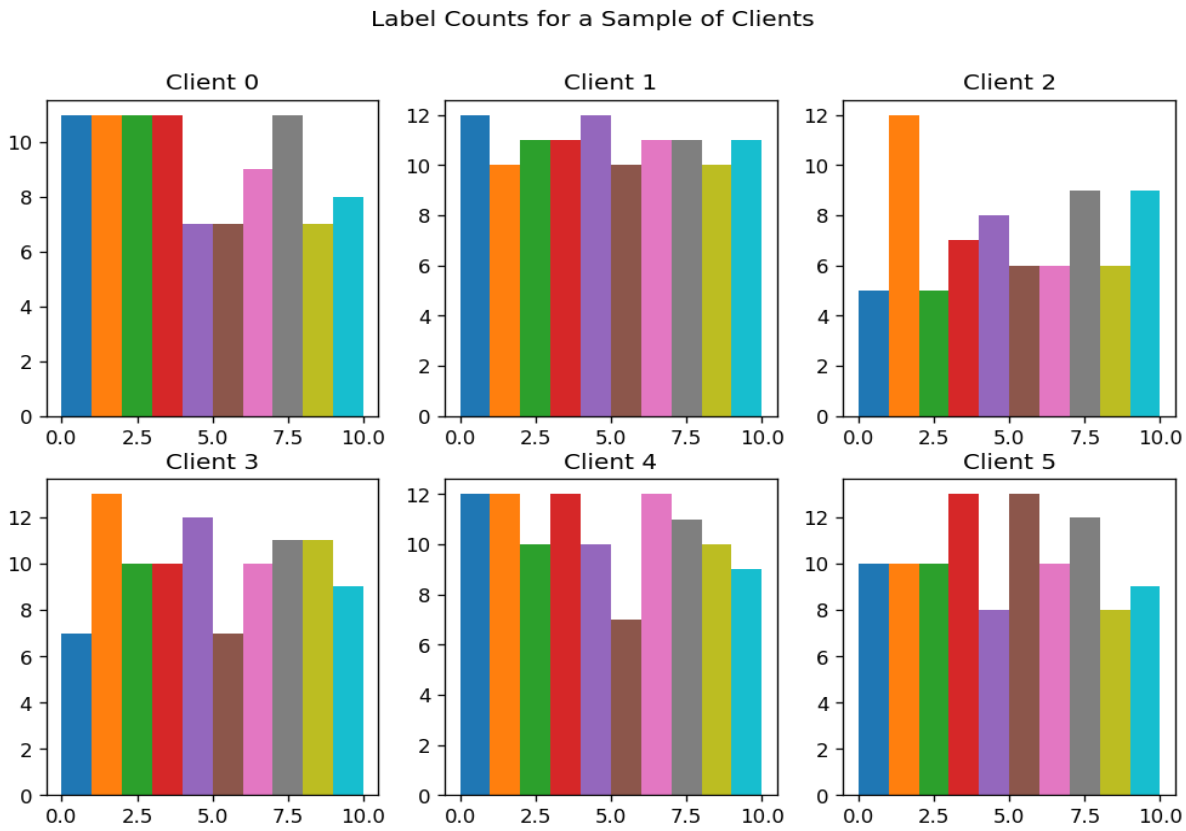



Figure 3.3: Label counts across clients

Per run, we fix a client split: approximately 90% training and 10% validation (client-level). Validation clients never participate in training or privacy spend; the test set is evaluated once at the end on the selected model.

### 3.4 Proposed Methodology

The Proposed Methodology builds upon the standard FedAvg algorithm and incorporates several key innovations to optimize the balance between privacy and model accuracy. These innovations include a 5-step high-to-low noise schedule, adaptive clipping, validation-based early stopping, and privacy accounting using RDP.

### 3.5 Model Architecture:

For our federated learning experiments, we used a fully connected neural network for image classification on the EMNIST dataset. The model consists of the following layers:

- **Input Layer:**Reshapes the 28×28 image input into a flat vector of length 784.
- **Hidden Layers:** Two fully connected layers of 200 units each, with ReLU activation to introduce non-linearity and allow for complex decision boundaries.

- **Output Layer:** A fully connected layer with 10 units corresponding to the 10 output classes of the EMNIST dataset, followed by SparseCategoricalCrossentropy loss (as we are working with integer-encoded labels).

The model was trained using Sparse Categorical Crossentropy loss with SparseCategoricalAccuracy as the metric for performance. This architecture was chosen to balance model simplicity with expressive power, making it well-suited for the EMNIST classification task under the federated learning framework.

### 3.5.1 Federated Averaging (FedAvg)

Federated Averaging (FedAvg)[20] is an algorithm used in Federated Learning to train a global model by averaging the model updates from multiple clients. The process proceeds as follows:

**Step 1: Initialization**

The central server initializes the global model $w_0$ and sends it to all clients.

**Step 2: Local Model Training**

Each client $k$ performs local training on its dataset $D_k$ for $E$ local epochs. Let the local model on client $k$ after $t$-th round of training be denoted as $w_k^{(t)}$.

$$w_k^{(t)} = w_0 - \eta \nabla F_k(w_0) \tag{3.1}$$

where $\eta$ is the learning rate, and $\nabla F_k(w_0)$ is the gradient of the local loss function $F_k$ on client $k$.

**Step 3: Model Aggregation**

After the local training, each client sends the updated model parameters $w_k^{(t)}$ to the central server. The server then aggregates these updates by averaging them weighted by the number of data points on each client:

$$w^{(t+1)} = \frac{1}{N} \sum_{k=1}^{N} n_k w_k^{(t)} \tag{3.2}$$

where $N$ is the total number of clients, $n_k$ is the number of data points on client $k$, and $w^{(t+1)}$ is the new global model sent back to the clients.

**Step 4: Repeat**

Steps 2 and 3 are repeated for several rounds until convergence.

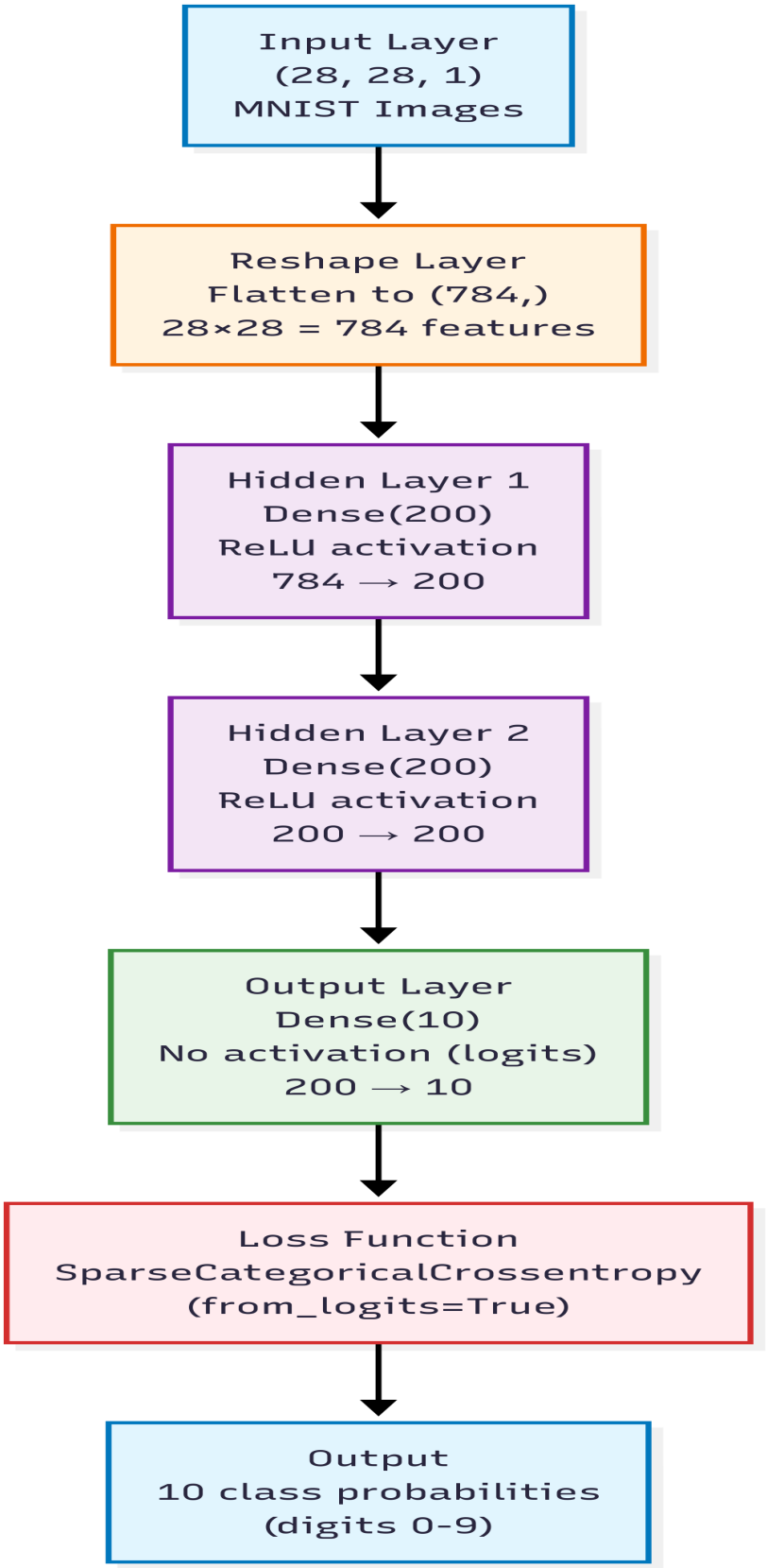



Figure 3.4: Model Architecture

### 3.5.2 Differential Privacy Mechanism

To ensure the privacy of clients' updates, we apply Differential Privacy (DP) using the Gaussian mechanism with adaptive clipping[21]. These operations are managed by the server side in TensorFlow Federated (TFF) as part of the model update aggregation process. Additionally, Poisson subsampling is used for client selection in each round, influencing the overall privacy budget.

**Client Selection and Poisson Subsampling:** We use Poisson subsampling over the training pool: each training client is independently included with nominal probability

$$q_{\text{nom}} = \frac{C_{\text{opt}}}{N}, \tag{3.3}$$

where $N$ is the total number of clients. For privacy accounting, we use the actual per-round participation

$$q_t = \frac{K_t}{|D_{\text{train}}|}. \tag{3.4}$$

If $K_t = 0$ (no participant), the round touches no data and incurs no privacy cost.

**Adaptive Clipping.** Before aggregation in round $t$, each client update is clipped to a *global* $\ell_2$ bound $C_t$:

$$\tilde{u}_i^{(t)} = u_i^{(t)} \cdot \min\left(1, \frac{C_t}{\|u_i^{(t)}\|_2}\right). \tag{3.5}$$

We employ an adaptive scheme that updates the single scalar $C_t$ across rounds using the distribution of observed update norms (e.g., targeting a chosen quantile), which improves stability under non-IID data. After clipping, the server adds Gaussian noise to the *sum* of clipped updates (with variance proportional to $C_t^2$) to ensure client-level differential privacy.

**Gaussian Noise:** After clipping the updates, Gaussian noise is added to the aggregated updates to guarantee privacy. The noise is calibrated such that the total privacy expenditure (measured in $\epsilon$ and $\delta$) is bounded by the target privacy parameters. The amount of noise is proportional to the clipping bound $C_t$, and is sampled from a Gaussian distribution with variance $\sigma_t^2$:

$$\hat{g}(t) = \frac{1}{K_t} \sum_{i \in S_t} \tilde{u}_i(t) + N(0, \sigma_t^2 I) \tag{3.6}$$

where $\tilde{u}_i(t)$ is the clipped update for client $i$, $K_t$ is the number of clients in round $t$, and $\sigma_t$ is the noise level determined by the noise scheduling. This approach ensures that the contribution of

each client to the global model is controlled based on the magnitude of their update, preserving privacy without excessively degrading model utility.

### 3.5.3 Calibration of Clients-per-Round $C_{\text{opt}}$ and Base Sigma $\sigma_0$

Let $N = |D_{\text{all}}|$ be the total number of available clients before the train/validation split, and let $T$ be the target number of rounds. We calibrate the integer clients-per-round parameter $C$ by modeling each round as a Poisson-subsampled Gaussian mechanism in which the inclusion rate and noise scale are tied by a fixed ratio $\lambda > 0$:

$$q \;=\; \frac{C}{N}, \qquad \sigma \;=\; \lambda\, C. \tag{3.7}$$

The per-round event is

$$\mathcal{E}_t \;=\; \mathrm{PoissonSubsample}(q) \;\circ\; \mathrm{Gaussian}(\sigma), \tag{3.8}$$

and the total Rényi DP cost over $T$ rounds is

$$\rho(\alpha) \;=\; \sum_{t=1}^{T} \rho_t(\alpha), \qquad \varepsilon(\delta) \;=\; \min_{\alpha>1} \frac{\rho(\alpha) + \ln(1/\delta)}{\alpha - 1}. \tag{3.9}$$

We search over integer $C \in \{1, \ldots, N\}$ and select

$$C_{\text{opt}} \;=\; \arg\min\{\, C \;:\; \varepsilon(\delta) \le \varepsilon \text{ under } T \text{ rounds with } q{=}C/N,\ \sigma{=}\lambda C \,\},$$

where $\varepsilon$ and $\delta$ are the target privacy parameters. The *base sigma* is then

$$\sigma_0 \;=\; \lambda\, C_{\text{opt}}. \tag{3.10}$$

**Notes.** (i) Calibration ties $\sigma$ to $C$ via $\sigma = \lambda C$ *only for the pre-split horizon* to obtain $C_{\text{opt}}$ and $\sigma_0$; during training, privacy accounting uses the *actual* $q_t = K_t/N_{\text{tr}}$. (ii) Rounds with $K_t = 0$ incur no privacy cost and are not composed. (iii) Gaussian noise is added to the clipped *sum* of client updates prior to normalization; clipping uses a single global $\ell_2$ bound $C_t$ per round.

**Algorithm 1** Calibration of $C_{\text{opt}}$ and Base Noise $\sigma_0$

**Input:** total clients $N$, rounds $T$, target budget $(\varepsilon^*, \delta)$, noise–clients ratio $\lambda$
**Output:** $C_{\text{opt}}$, $\sigma_0$, nominal sampling $q_{\text{nom}}$
Define event factory: for integer $C \in \{1, \ldots, N\}$, set $q \leftarrow C/N$, $\sigma \leftarrow \lambda C$, and per-round event $\mathcal{E}(q, \sigma) \leftarrow \text{PoissonSubsample}(q) \circ \text{Gaussian}(\sigma)$
Compose privacy over $T$ rounds with an RDP accountant to obtain $\varepsilon(C; \delta)$ for $\mathcal{E}(q, \sigma)$
Search over integer $C$ (e.g., bracketed/binary search) to find the smallest $C$ s.t. $\varepsilon(C; \delta) \leq \varepsilon^*$
Set $C_{\text{opt}} \leftarrow C$, $\sigma_0 \leftarrow \lambda\, C_{\text{opt}}$, $q_{\text{nom}} \leftarrow C_{\text{opt}}/N$
**Return:** $C_{\text{opt}}$, $\sigma_0$, $q_{\text{nom}}$

### 3.5.4 Noise Scheduling

The core contribution to my proposed methodology is the noise schedulling. We experiment with multiple schedules for noise multiplier $\sigma_t$:

- **Flat Noise:** The noise level $\sigma_t$ is constant throughout all rounds, providing uniform privacy across the training process.
- **Low-to-High Noise:** The noise starts low and gradually increases, consuming more $\epsilon$ early in the training process.
- **High-to-Low Noise:** The noise starts high and gradually decreases, providing stronger privacy protection early in training when the model is unstable and reducing noise as the model converges. This schedule minimizes privacy cost during fine-tuning, which is when the model benefits most from precise updates. The 5-step High-to-Low schedule consists of five equally spaced steps in the training rounds. The noise multiplier $\sigma_t$ is set to:

$$\sigma_t = m_s.\sigma_0, m = [2.0, 1.75, 1.5, 1.25, 1.0] \tag{3.11}$$

  where $m_s$ is the multiplier for step s and $\sigma_0$ is the base sigma. The scheduler updates $\sigma_t$ at each step boundary, ensuring efficient use of privacy across the rounds.

**Algorithm 2** Noise Scheduling Strategy

**Input:** round $t \in \{0, \ldots, T\}$, horizon $T$, base noise $\sigma_0$, mode $\in$ {flat, step5_high $\rightarrow$ low, step5_low $\rightarrow$ high, cosine, linear_decreasing, exponential_decay}
**Output:** multiplier $m_t$, per-round noise $\sigma_t = m_t \sigma_0$
**Case flat:** $m_t \leftarrow 1$
**Case step5_high→low:** $s \leftarrow \min(\lfloor 5t/T \rfloor, 4)$; $m_t \leftarrow [2.0, 1.75, 1.5, 1.25, 1.0][s]$
**Case step5_low→high:** $s \leftarrow \min(\lfloor 5t/T \rfloor, 4)$; $m_t \leftarrow [1.0, 1.25, 1.5, 1.75, 2.0][s]$
**Case cosine (high→low):** $m_t \leftarrow 1 + \frac{1}{2}\big(1 + \cos(\pi t/T)\big)$
$\sigma_t \leftarrow m_t \cdot \sigma_0$
**Note:** If $\sigma_t \neq \sigma_{t-1}$ at a step boundary, rebuild the DP aggregator with $\sigma_t$ and warm-start the model.

### 3.5.5 RDP Accounting and $\epsilon$ Reporting

For each round $t$, the mechanism is

$$\mathcal{E}_t = \text{PoissonSubsample}(q_t) \circ \text{Gaussian}(\sigma_t). \tag{3.12}$$

Let $\rho_t(\alpha)$ be its Rényi DP at order $\alpha > 1$. Over $T$ rounds:

$$\rho(\alpha) = \sum_{t=1}^{T} \rho_t(\alpha), \quad \epsilon(\delta) = \min_{\alpha>1} \left( \rho(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1} \right). \tag{3.13}$$

We maintain a running accountant and report $\epsilon_t$ at $\delta = 10^{-5}$. Only rounds with $K_t > 0$ are composed.
Training stops when (i) the validation metric does not improve for p evaluations (patience), or (ii) the running accountant reaches ($\epsilon_{\text{max}} = 3.0$); we report the best-val-accuracy checkpoint along with its ($\epsilon$).

### 3.5.6 Evaluation protocols

The effectiveness of the proposed methodology is evaluated using the following metrics:

- Validation Accuracy at different ($\epsilon$) levels.
- iso ($\epsilon$) at ($\epsilon$) = 3, to compare rounds needed to utilize the privacy budget and accuracy acquired.
- Normalized AUC of the ValAcc vs ($\epsilon$) curve, measuring privacy-efficiency.
- Accuracy vs ($\epsilon$) and rounds yo ($\epsilon$) comparisons across methods (flat, low-to-high, high-to-low).

These metrics are compared across three schedules to understand the trade-off between privacy and utility in each configuration.

**Algorithm 3** Federated Learning with Differential Privacy and Noise Scheduling

**Input:** clients $S = \{C_1, \ldots, C_N\}$, sampling ratio $q$, budget $(\varepsilon^*, \delta)$, rounds $T$, local epochs $K$, base noise $\sigma_0$, multipliers $m_t$, clip bound $C_t$
**Output:** global model $w_T^*$, privacy loss $\varepsilon_T$
Initialize $w_0$, accountant $\varepsilon_0 = 0$
**for** $t = 1$ to $T$ **do**
  Compute $\sigma_t = m_t \cdot \sigma_0$
  Sample $Z_t \subseteq S$ with prob. $q$, set $K_t = |Z_t|$
  **if** $K_t = 0$ **then**
    **continue**
  **end if**
  **for** each $i \in Z_t$ **in parallel do**
    Local update $u_i^{(t)}$ via $K$ epochs on $D_i$
    Clip: $\tilde{u}_i^{(t)} = u_i^{(t)} \cdot \min\left(1, \frac{C_t}{\|u_i^{(t)}\|_2}\right)$
  **end for**
  Aggregate: $\hat{g}^{(t)} = \frac{1}{K_t} \sum_{i \in Z_t} \tilde{u}_i^{(t)} + \mathcal{N}(0, \sigma_t^2 I)$
  Update: $w_t \leftarrow w_{t-1} + \hat{g}^{(t)}$
  Accountant: compose $\mathcal{E}_t = \mathrm{PoissonSubsample}(q_t) \circ \mathrm{Gaussian}(\sigma_t)$
  Report $\varepsilon_t$; if $\varepsilon_t \geq \varepsilon_{\max}$ or early stop $\rightarrow$ break
**end for**
**Return:** $w_T^*, \varepsilon_T$

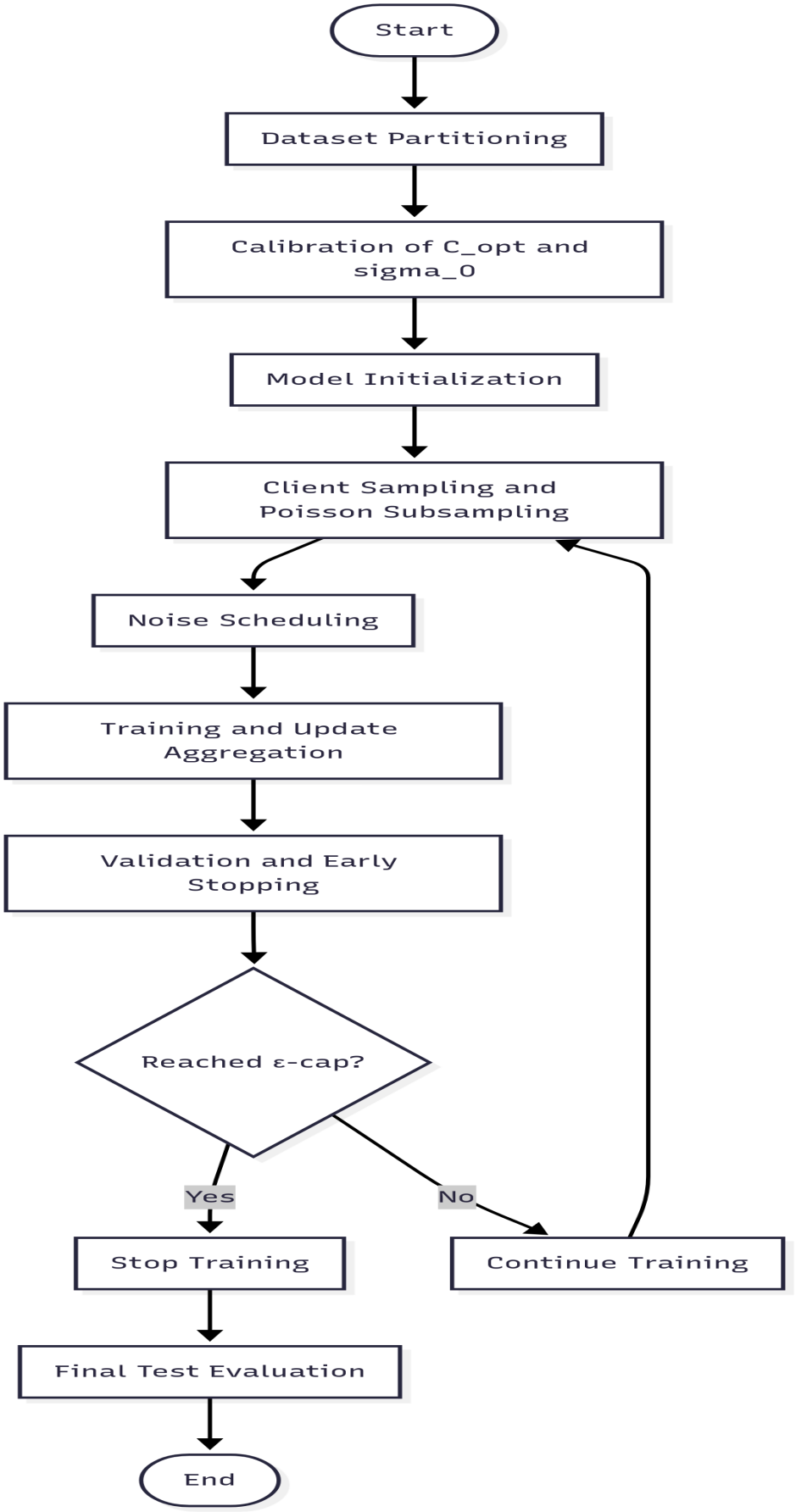



Figure 3.5: Flowchart of Proposed Methodology

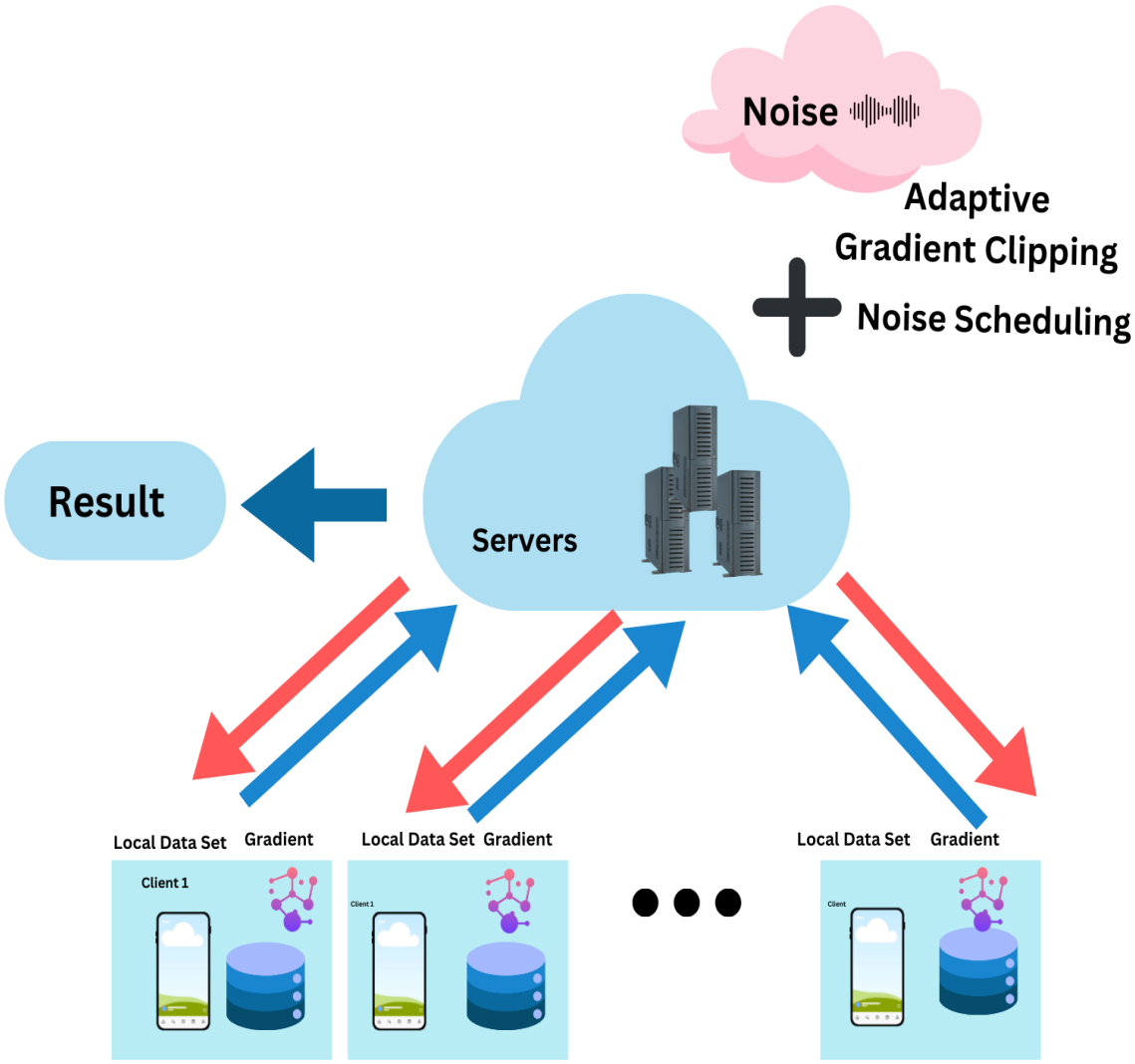



Figure 3.6: Illustration of our Proposed Methodology

Table 3.1: Comparison of Methodologies: Ours vs. Existing Approaches

| **Method** | **Key Components** | **Limitations in Methodology** | **Our Advantages** |
|---|---|---|---|
| **NbAFL [22]** | Noise added before aggregation, central DP | Works only at very weak privacy budgets ($\epsilon \approx 100$); accuracy collapses at strong DP | We operate at realistic budgets ($\epsilon \leq 3$) while preserving accuracy with adaptive clipping and noise scheduling |
| **FL-ODP [12]** | Fixed clipping, optimized sampling, constant Gaussian noise | Evaluated on MNIST with 10 clients; non-IID setting mild; no adaptive clipping or per-round scheduling | We combine adaptive clipping with High→Low noise scheduling, validated on harsh EMNIST writer-level non-IID with 3000+ clients |
| **Adaptive DP-FL [23]** | Adaptive clipping with decaying noise schedule | Shard-based split with only 10 clients; moderate non-IID; noise decay not tied to privacy accountant | We use large-scale Poisson subsampling and strict RDP accounting, ensuring transparent $\epsilon$ logs at each round |
| **DDPFL [18]** | Differentiated noise per parameter (importance coefficients) | Parameter-wise tuning only; no round-wise scheduling; no adaptive clipping | We coordinate both levers: round-wise noise schedule + adaptive clipping to match dynamic client updates |
| **Google DP-FedAvg [14]** | Production FL with adaptive clipping and moments accountant | Large-scale system but no structured noise schedule; focus is production feasibility, not methodological trade-offs | We extend adaptive clipping with systematic scheduling and iso-$\epsilon$ comparisons, emphasizing methodology-level contribution |
| **Bu et al. (GDP) [16]** | Adaptive noise scaling across training rounds (centralized) | Centralized setting only; not federated; no adaptive clipping | We transfer the scheduling idea into FL and combine it with adaptive clipping under realistic non-IID conditions |
| **Wu et al. [17]** | Privacy budget allocation in convex ML (vary noise per round) | Limited to convex models; synthetic datasets; not federated | We provide empirical deep-learning evidence in FL on EMNIST non-IID |
| **Our Method (Proposed)** | FedAvg + Adaptive clipping + 5-step High→Low noise schedule + Poisson subsampling + RDP accounting | Needs careful calibration of $C_{opt}$ and $\sigma_0$; slightly lower raw accuracy vs. flat schedule | Achieves best privacy–utility trade-off (up to 24.6% higher Acc/$\epsilon$), transparent per-round $\epsilon$ tracking, scalable to thousands of clients with harsh non-IID |

# CHAPTER IV

# Implementation, Results, and Discussions

## 4.1 Introduction

This section describes the practical implementation of the methodology introduced in Section 3. The goal is to translate the proposed approach into a working federated learning system and to evaluate its effectiveness under different settings. We begin by outlining the experimental environment, datasets, and model architecture. Then, we present the details of our implementation for both standard federated learning (FedAvg) and differentially private federated learning (DP-FL). Particular emphasis is placed on the integration of adaptive clipping, privacy calibration, and noise scheduling strategies. Finally, we define the evaluation protocols, present the experimental results, and analyze whether the objectives of the research have been achieved.

## 4.2 Experimental Setup

- Environment The experiments were implemented and executed on **Google Colab**. The environment configuration was as follows:
  - **Platform:** Google Colab (cloud runtime)
  - **Programming language:** Python 3.11
  - **Frameworks:** TensorFlow 2.18.0, TensorFlow Federated (TFF) 0.87.0
  - **Hardware:** NVIDIA T4 GPU (when available); otherwise CPU fallback

  This configuration ensured reproducibility while leveraging GPU acceleration for most experiments, with automatic fallback to CPU when GPU resources were unavailable.
- **Datasets** Two benchmark datasets were used to evaluate the proposed methodology:
  - **MNIST:**
    - Standard handwritten digits dataset (60,000 training, 10,000 test images).
    - Used for baseline experiments under both IID and non-IID partitions.
    - Non-IID splits were created by restricting each client to a subset of digit classes.
  - **EMNIST (ByWriter):**
    - Real-world cross-device dataset partitioned by writer identity.

    * Provides highly non-IID distributions with significant label skew and client heterogeneity.
    * Total of 3383 clients (writers).
    * Partitioned into 3045 training clients (90%), 338 validation clients (10%), and the remaining clients used as a held-out test set.
    * The test set was evaluated only once after training to ensure unbiased performance reporting.

- **4.2.3 Model Architecture** A fully connected neural network was used as the base model to balance computational simplicity with sufficient expressive power. The architecture was as follows:

  – **Input Layer:** Reshapes each $28 \times 28$ image into a $784$-dimensional vector.

  – **Hidden Layers:** Two dense layers with 200 units each, using ReLU activation.

  – **Output Layer:** Dense layer with 10 units, corresponding to digit classes, followed by softmax activation.

  – **Loss Function:** Sparse Categorical Crossentropy.

  – **Evaluation Metric:** Sparse Categorical Accuracy.

  This lightweight design allowed efficient federated training on resource-constrained environments while retaining enough capacity to capture handwritten digit patterns.

Table 4.1: Hyperparameters on MNIST (IID and non-IID).

| **Hyperparameter** | **FedAvg (no DP)** | **DP-FedAvg** |
|---|---|---|
| Clients (simulated) | 50 | 50 |
| Partition | IID / non-IID | IID / non-IID |
| Batch size | 20/64 | 20/64 |
| Local epochs ($K$) | 5 | 5 |
| Total rounds ($T$) | 100/80 | 100/80 |
| Client learning rate | 0.01 | 0.01 |
| Optimizer | SGD | SGD |
| Model | CNN | Same |
| DP $\delta$ | — | $10^{-5}$ |
| Noise multiplier $\sigma$ | — | $0.7$/$0.9$ |
| Privacy accounting | — | RDP (Poisson-subsampled Gaussian) |
| Aggregation | FedAvg | DP-FedAvg |

### 4.3 Implementation of FL and DP-FL

#### 4.3.1 Baseline Federated Learning (FedAvg)

We first implemented the standard **Federated Averaging (FedAvg)** algorithm in TensorFlow Federated (TFF). This served as a baseline for comparison with differentially private extensions. Experiments were conducted on:

- **MNIST IID:** clients received randomly shuffled data partitions, simulating homogeneous distributions.
- **MNIST non-IID:** clients were assigned a limited subset of digit classes, introducing label skewness.

This baseline allowed us to observe the impact of distribution heterogeneity before introducing privacy constraints. Differentially Private Federated Learning (DP-FL) To enforce client-level privacy, we integrated the Gaussian differential privacy mechanism into FedAvg using TFF's dp_aggregator. Key components include:

- **Poisson subsampling:** each client was selected with probability $q = C_{\text{opt}}/N$ per round.
- **Gradient clipping:** per-client updates were clipped to bound sensitivity.
- **Gaussian noise:** noise scaled by $\sigma_t$ was added to aggregated updates.
- **Privacy accounting:** RDP was used to track $(\varepsilon, \delta)$ across training rounds.

We compared DP-FL against baseline FedAvg on both MNIST IID and non-IID partitions.

**MNIST IID: FedAvg vs DP-FL** Figure 4.1 shows the test accuracy over 100 rounds for FedAvg (no privacy) and DP-FedAvg with Gaussian noise ($\sigma = 0.7$). As expected, the addition of differential privacy introduces a performance gap due to noise perturbation of client updates.

- **FedAvg (no DP):** reached a final test accuracy of $97.33\%$.
- **DP-FedAvg ($\sigma = 0.7$):** reached a final test accuracy of $94.0\%$.

Although accuracy decreases under DP, the model still converges stably, demonstrating the feasibility of applying client-level differential privacy in federated learning.

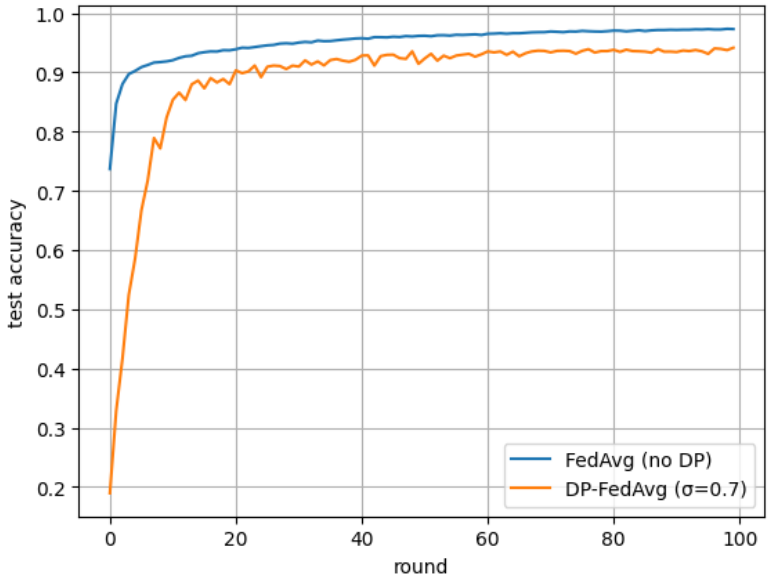



Figure 4.1: Test accuracy of FedAvg (no DP) vs DP-FedAvg ($\sigma = 0.7$) on MNIST IID dataset.

**MNIST non-IID: FedAvg vs DP-FL** Figure 4.2 shows the test accuracy over 80 rounds for FedAvg (no privacy) and DP-FedAvg with Gaussian noise ($\sigma = 0.9$). As expected, the addition of differential privacy introduces a performance gap due to noise perturbation of client updates.

- **FedAvg (no DP):** reached a final test accuracy of $80.87\%$.
- **DP-FedAvg ($\sigma = 0.9$):** reached a final test accuracy of $53.81\%$.

Although accuracy decreases under DP, the model still converges stably, demonstrating the feasibility of applying client-level differential privacy in federated learning.

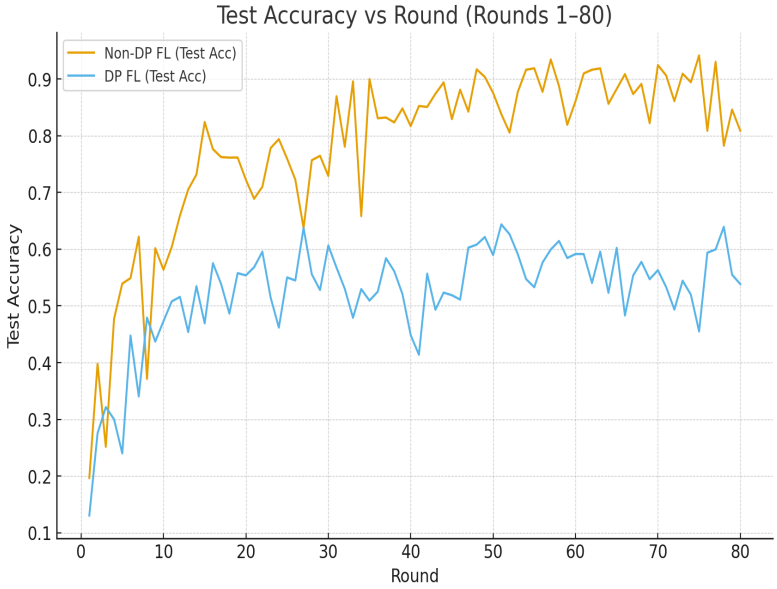



Figure 4.2: Test accuracy of FedAvg (no DP) vs DP-FedAvg ($\sigma = 0.9$) on MNIST non-IID dataset.

### 4.3.2 EMNIST Experiments

The main experiments were conducted on **EMNIST (ByWriter)**, which introduces strong non-IID features:

- Total of 3383 clients (writers).
- Split into 3045 training clients (90%) and 338 validation clients (10%).
- The remaining clients formed a held-out test set, evaluated once at the end.

Table 4.2: Hyperparameters on EMNIST (ByWriter) and DP calibration.

| **Hyperparameter** | **FedAvg (no DP)** | **DP-FedAvg** |
|---|---|---|
| Total clients (writers) | 3383 | |
| Training / Validation split | 3045 (90%) / 338 (10%) | |
| Test usage | 312 batch; evaluated once at end | |
| Batch size | 32 | 32 |
| Local epochs ($K$) | 5 | 5 |
| Total rounds ($T$) | 100 | 100–150 |
| Client learning rate | 0.01 | 0.01 |
| Server learning rate | 1.0 | 1.0 |
| Server optimizer | SGDM (momentum 0.9 | SGDM (momentum 0.9) |
| Model | MLP (200–200–10) | same |
| Clipping | — | Adaptive global $\ell_2$ bound $C_t$ |
| DP $\delta$ | — | $10^{-5}$ |
| $\varepsilon$ cap (iso-$\varepsilon$) | — | 3.0 |
| Calibration (target $\varepsilon$=3.0) | — | $C_{\text{opt}}$=90 |
| Base noise | — | $\sigma_0$=0.9 |
| Sampling ratio | — | $q$=$C_{\text{opt}}/N \approx 0.0266$ |
| Noise schedules | — | Flat; Step5 Low→High; Step5 High→Low |
| Privacy accounting | — | RDP (Poisson-subsampled Gaussian) |
| Aggregation | FedAvg | DP-FedAvg |

We implemented both FedAvg and DP-FL on EMNIST to assess performance under realistic cross-device heterogeneity and to evaluate and set the baseline for our experiment. **EMNIST: FedAvg vs DP-FedAvg** Figure 4.3 shows the final test metrics on EMNIST (ByWriter) for FedAvg (no DP) and DP-FedAvg.

- **FedAvg (no DP):** Accuracy = 83.2%, Loss = 0.548
- **DP-FedAvg:** Accuracy = 80.0%, Loss = 0.638

Although the introduction of differential privacy results in a modest decrease in accuracy (3.2 percentage points) and slightly higher loss, the model still achieves competitive performance

under strong client-level privacy guarantees. This highlights that the privacy–utility trade-off remains manageable even in a challenging non-IID cross-device setting.

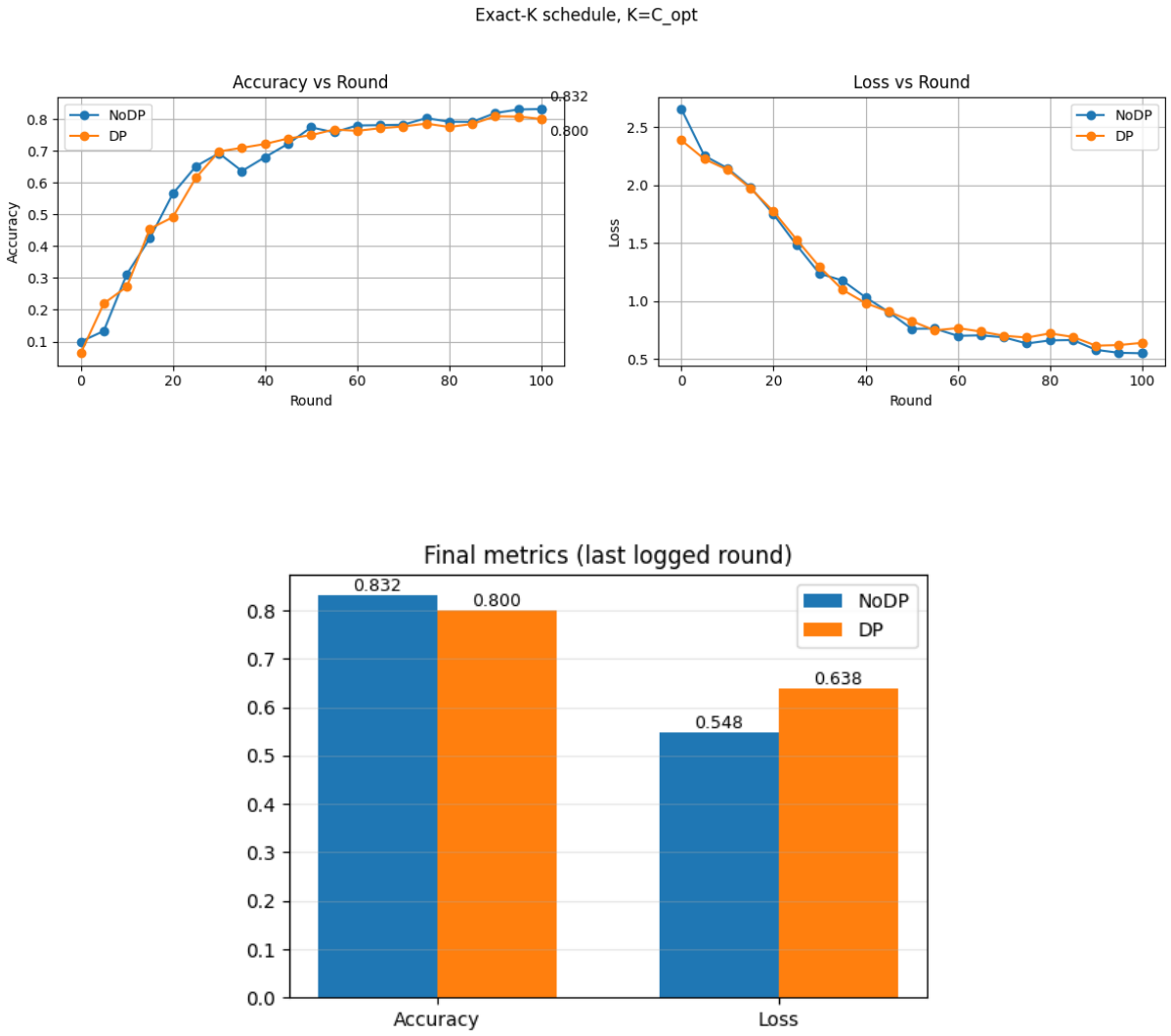



Figure 4.3: Final metrics (Test accuracy and loss) on EMNIST (ByWriter) with FedAvg and DP-FedAvg.

## 4.4 Implementation of our proposed Method

In this section, we describe the practical implementation of our proposed methodology. The entire pipeline was built using Python 3.11 with TensorFlow 2.18.0 and TensorFlow Federated 0.87.0 on Google Colab. Our implementation follows the design outlined in Chapter 3, extending the standard FedAvg algorithm with differential privacy mechanisms, adaptive clipping, and dynamic noise scheduling. Additional features such as validation-based early stopping and iso- termination were incorporated to evaluate the privacy–utility trade-off more systematically.

### 4.4.1 Adaptive Clipping

To improve stability under DP noise, we applied **adaptive clipping**, where the clipping bound $C_t$ was dynamically adjusted based on update norms. This ensures that large updates are clipped more aggressively while smaller updates remain less affected. We compared EMNIST results with and without adaptive clipping.

### 4.4.2 Noise Scheduling Strategies

We evaluated multiple strategies for adjusting the noise multiplier $\sigma_t$ across rounds:

- **Flat noise:** constant $\sigma_t$ for all rounds.
- **Low-to-High:** $\sigma_t$ gradually increased over training.
- **High-to-Low:** $\sigma_t$ started high and decreased in steps, our proposed method.
- (Optional) **Cosine:** follows cosine schedule.

Each strategy was tested on EMNIST with DP-FL, and their privacy-utility trade-offs were compared.

### 4.4.3 5 Step High-to-Low noise scheduling

Our proposed method introduces a **5-step High→Low noise schedule**, designed to balance privacy and accuracy in differentially private federated learning. Training is divided into five equal segments, where the Gaussian noise multiplier $\sigma_t$ decreases stepwise as:

$$\sigma_t = m_s \cdot \sigma_0, \quad m_s \in \{2.0,\ 1.75,\ 1.5,\ 1.25,\ 1.0\}, \tag{4.1}$$

with $\sigma_0$ denoting the base noise level obtained from privacy calibration. In the early rounds, higher noise is applied to provide stronger privacy when the model is unstable. As training progresses and the model approaches convergence, the noise is gradually reduced, enabling more precise updates during fine-tuning.

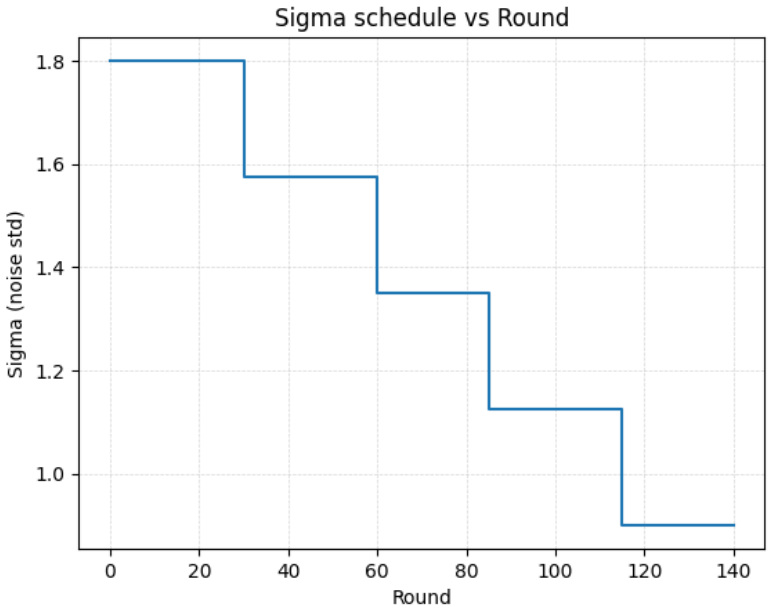



Figure 4.4: Sigma schedule over the round of the proposed 5-step High→Low schedule.

**Calibration.** With target $(\varepsilon{=}3.0, \delta{=}10^{-5})$ over 150 rounds, the accountant selected $C_{\text{opt}}{=}90$ clients/round, giving base $\sigma_0{=}0.9$ and nominal $q{=}C_{\text{opt}}/N{=}0.0266$.

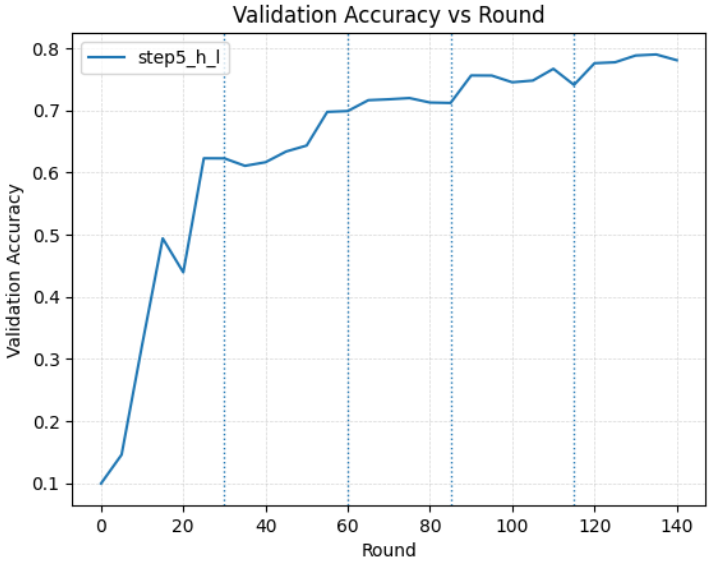



Figure 4.5: Validation accuracy over rounds using the proposed 5-step High→Low schedule on EMNIST (ByWriter).

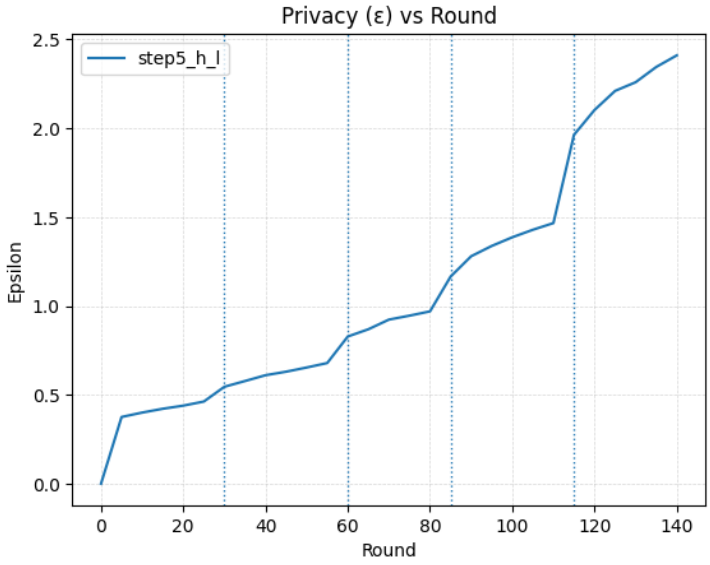



Figure 4.6: Validation loss over rounds using the proposed 5-step High→Low schedule on EMNIST (ByWriter).

The proposed 5-step High→Low noise schedule was executed for a maximum of $T = 140$ rounds under a privacy budget of $\varepsilon_{\max} = 3.0$. Training converged earlier, stopping at $\varepsilon = 2.41$ with $\delta = 10^{-5}$. The final test accuracy reached $78.5\%$ with a test loss of $0.705$, while the best validation accuracy achieved during training was $78.9\%$.

This demonstrates that the High→Low schedule was able to approach 80% accuracy without fully consuming the privacy budget, highlighting its efficiency in balancing privacy and utility.This confirms that injecting stronger noise at the beginning and reducing it later provides a favorable privacy–utility trade-off, making the proposed schedule more effective than uniform noise injection.

Table 4.3: Final results of proposed High→Low noise schedule (EMNIST).

| Metric | Value |
|---|---|
| Final round ($T$) | 140 |
| Target $\varepsilon$ | 3.00 |
| Final $\varepsilon$ | 2.41 |
| $\delta$ | $10^{-5}$ |
| Best validation accuracy | 78.9% |
| Final test accuracy | 78.5% |
| Final test loss | 0.705 |

**Calibration.** With target $(\varepsilon{=}3.0, \delta{=}10^{-5})$ over 100–150 rounds, the accountant selected $C_{\text{opt}}{=}90$ clients/round, giving base $\sigma_0{=}0.9$ and nominal $q{=}C_{\text{opt}}/N{=}0.0266$.

### 4.4.4 Other Noise Scheduling Implementations

In addition to the proposed 5-step High→Low schedule, we implemented three baseline noise scheduling strategies for comparison:

- **Flat schedule:** constant noise multiplier $\sigma_t = \sigma_0$ for all rounds. This represents the standard DP-FL setup with uniform privacy cost allocation.
- **Low→High schedule:** noise multiplier increases across training, with multipliers

$$m_s \in \{1.0,\ 1.25,\ 1.5,\ 1.75,\ 2.0\}, \tag{4.2}$$

  making training more precise early but progressively noisier.

- **Cosine schedule:** noise multiplier follows a cosine decay:

$$\sigma_t = \sigma_0 \cdot (1.0 + 0.5 \cdot (1 + \cos(\pi t/T))), \tag{4.3}$$

  producing a smooth transition from high to low noise.

All schedules were integrated into the same TFF pipeline by reinitializing the DP aggregation factory whenever $\sigma_t$ changed. This ensured that each schedule was evaluated under identical conditions, differing only in how noise was allocated across rounds.

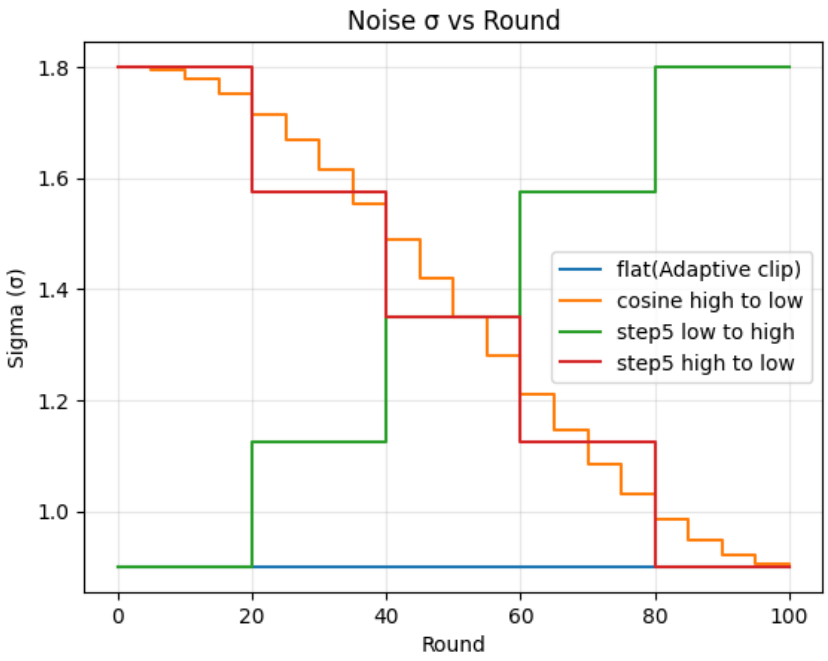



Figure 4.7: Noise scheduling patterns across rounds for different strategies..

*Implementation note.* The cosine schedule was implemented by re-initializing the DP aggregation factory whenever $\sigma_t$ changed. Because $\sigma_t$ varies every round, this required a costly re-aggregation step at each iteration. By contrast, the 5-step schedules (Low→High and High→Low) only require four re-initializations in total, making them far more efficient in practice. For this reason, the cosine schedule was retained only for completeness and dropped from later validated experiments.

#### 4.4.5 Summary of Implemented Variants

For clarity, the experiments can be summarized as:

- **MNIST IID:** FedAvg, DP-FL (flat noise).
- **MNIST non-IID:** FedAvg, DP-FL (flat noise).
- **EMNIST:** FedAvg, DP-FL (flat, Low→High, cosine, High→Low), with adaptive clipping.

This structured implementation enabled a systematic evaluation of the proposed methodology.

### 4.5 Experiment Design

To evaluate the proposed methodology, we implemented three categories of experiments in TensorFlow Federated (TFF). Each experiment was executed under the same federated learning setup, differing only in the treatment of validation, stopping rules, and privacy accounting:

1. **Non-validation runs (100 rounds).** Flat, Low→High, and High→Low and cosine schedules were executed for $T = 100$ rounds without validation clients or early stopping. These experiments establish baseline convergence behavior and raw accuracy–privacy trade-offs.
2. **Validation-enabled runs (up to 150 rounds).** A 90/10 split of clients into training and validation pools was applied. Flat, Low→High, and High→Low schedules were trained with:
    - **Validation tracking:** accuracy/loss recorded every 5 rounds.
    - **Early stopping:** stop if validation accuracy fails to improve for 3 consecutive evaluations ($\Delta = 0.002$).
3. **Iso-$\varepsilon$ runs.** To enable fair comparison under identical privacy budgets, we conducted iso-$\varepsilon$ experiments. Here, training was forced to stop exactly when the accountant reported $\varepsilon = 3.0$, or after 140 rounds if the budget was not reached. Final test accuracy and loss were reported for Flat, Low→High, and High→Low schedules.

This structured experimental design ensures that results are evaluated both under unconstrained conditions (non-validation runs) and under realistic privacy constraints (validation-enabled and iso-$\varepsilon$ runs).

### 4.6 Evaluation Metrics

We assess privacy–utility with the following measurements:

- **Accuracy vs. rounds** and **loss vs. rounds** (convergence).
- **Accuracy vs.** $\varepsilon$ computed with an RDP accountant (privacy–utility curve).
- **AUC(Acc vs** $\varepsilon$**)** as a privacy-efficiency score (higher is better).
- **Iso-**$\varepsilon$ **comparison**: accuracy at a fixed privacy budget (we use $\varepsilon \approx 2.3$ and $\varepsilon = 3.0$).
- **Stopping rules**: (i) early stopping on validation (patience=3, $\Delta = 0.002$); (ii) $\varepsilon$-cap stop (terminate when $\varepsilon \geq \varepsilon_{\max}$).

Validation clients never participate in training or privacy spend. The test set is evaluated once at the end.

### 4.7 Results and Analysis

This section presents the results of our experiments. We begin with analyzing the performance of different noise scheduling strategies in both non-validation and validation-enabled runs, and finally evaluate iso-$\varepsilon$ experiments.

#### 4.7.1 Noise Schedules without Validation (100 rounds)

We first trained DP-FL models for 100 rounds without validation clients or early stopping. This experiment shows the raw convergence behavior of different noise scheduling strategies before applying validation-based stopping rules.

**Illustrative Example: Privacy–Utility Pareto Curve** To illustrate how privacy and utility evolve together, Figure 4.12 shows a Pareto curve for one schedule, with accuracy annotated by training round. This highlights the trade-off between model accuracy and cumulative privacy cost ($\varepsilon$).

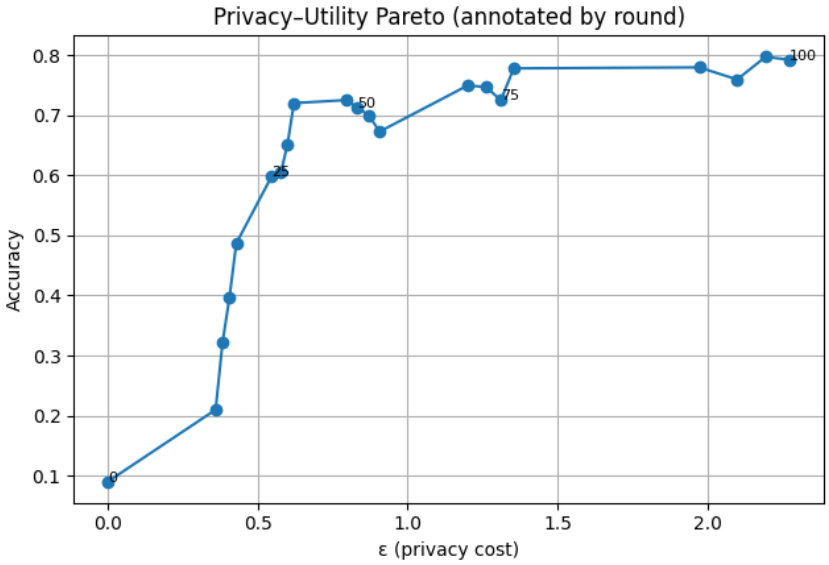
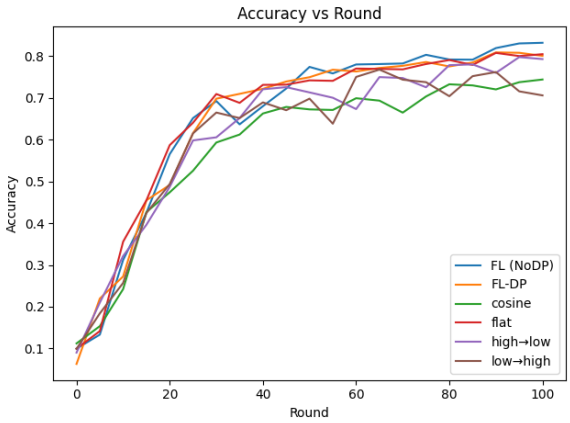
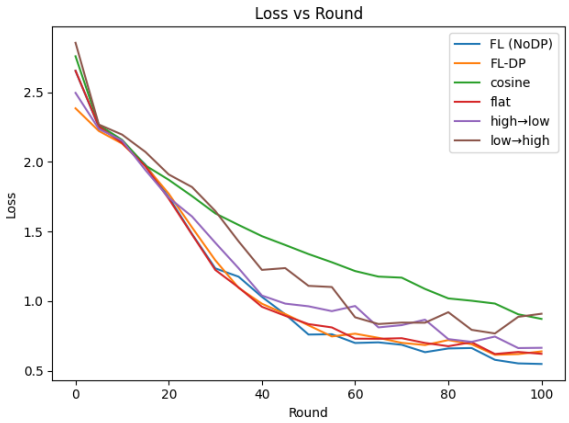



Figure 4.8: Privacy–utility Pareto curve annotated by round. Early rounds consume less privacy but yield rapid gains in accuracy.

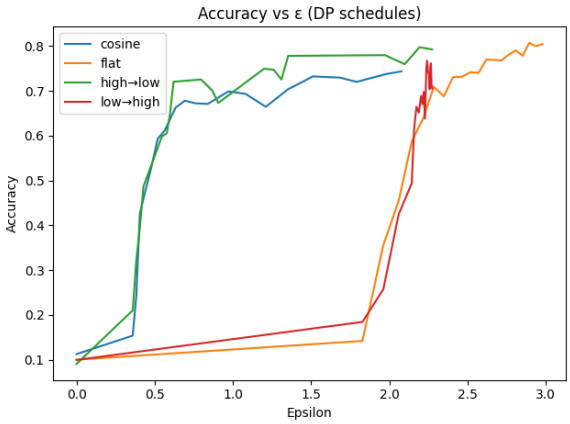
**Combined Results: Schedule Comparisons** Next, we compare all schedules directly.



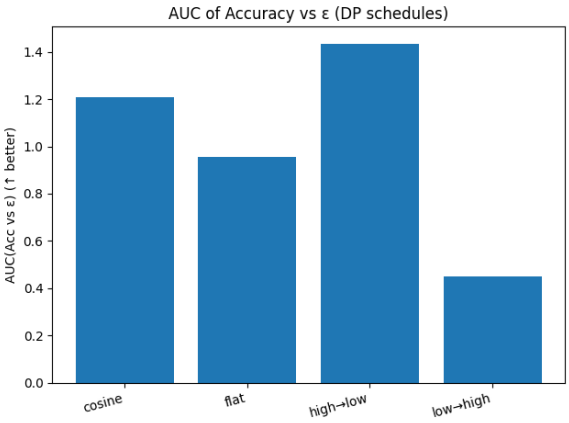

(a) Accuracy vs Round




(b) Loss vs Round




(c) Accuracy vs $\varepsilon$




(d) AUC(Acc vs $\varepsilon$)

Figure 4.9: Non-validation runs (100 rounds): Comparison of schedules across different metrics.

Table 4.4: Non-validation runs (100 rounds): schedule comparison.

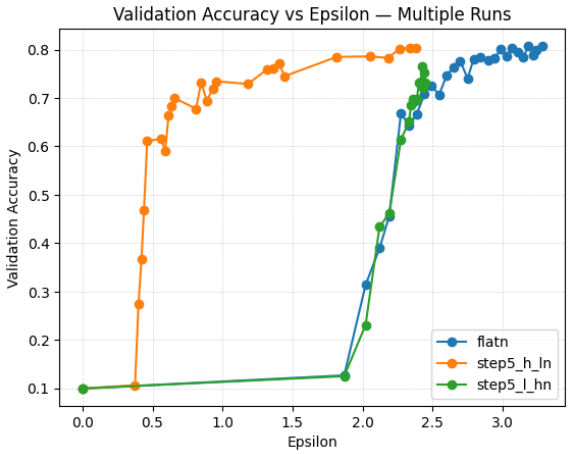
| Run | Best Acc | Best Round | Last Acc | Last Loss | AUC(Acc vs $\varepsilon$) | Last $\varepsilon$ |
|---|---|---|---|---|---|---|
| FL (NoDP) | 0.8315 | 100 | 0.8315 | 0.5480 | – | – |
| FL-DP | 0.8085 | 90 | 0.8002 | 0.6385 | – | 2.98 |
| Cosine | 0.7437 | 100 | 0.7437 | 0.8719 | 1.208 | 2.08 |
| Flat | 0.8073 | 90 | 0.8043 | 0.6209 | 0.954 | 2.98 |
| High→Low | 0.7972 | 95 | 0.7925 | 0.6642 | **1.434** | 2.27 |
| Low→High | 0.7673 | 65 | 0.7056 | 0.9090 | 0.448 | 2.27 |

High→Low delivers the best privacy efficiency: AUC(Acc vs $\varepsilon$) is **+50.4% higher** than Flat (1.434 vs 0.954), while Low→High is **53% worse** than Flat. Overall, these results confirm that allocating stronger noise early (High→Low) leads to the most favorable privacy–utility trade-off among the tested schedules. This shows that reducing noise as training progresses is more effective than increasing it.

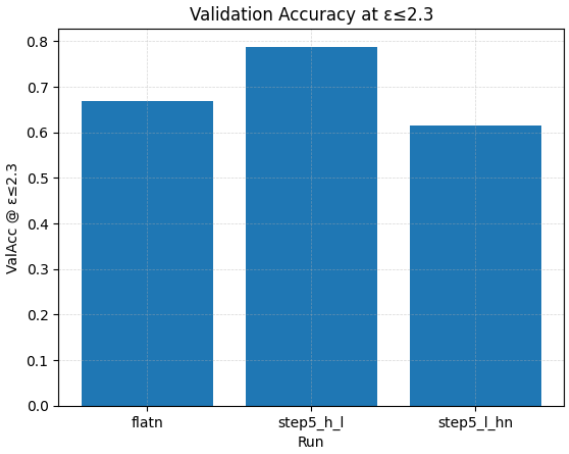
*Note on Cosine schedule.* While the cosine schedule achieved moderate accuracy (74.4%) and a reasonable AUC(Acc vs $\varepsilon$) of 1.21, it required re-building and re-aggregating the FL process at **every round**, making it significantly more computationally expensive compared to the 5-step schedules. Since our goal is to design efficient and practical DP-FL training, we did not include cosine in later validation-enabled and iso-$\varepsilon$ experiments.

### 4.7.2 Noise Scheduling with Validation (3 runs, $T = 140$)

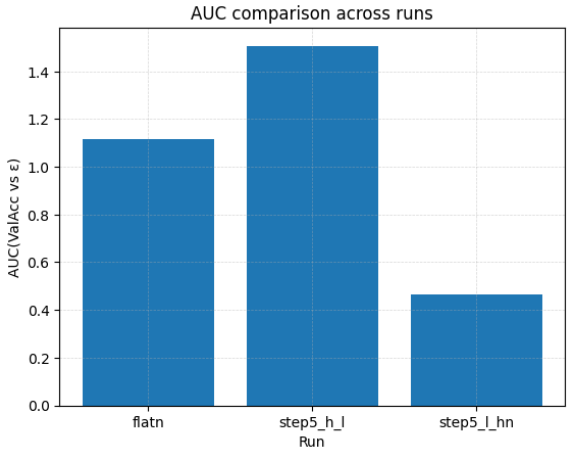
To better understand the privacy–utility trade-offs, we extended the experiments with validation-enabled runs under three schedules: *Flat*, *Low→High*, and *High→Low*. Validation clients were held out and early stopping was enabled to prevent overfitting. The training horizon was capped at 140 rounds.



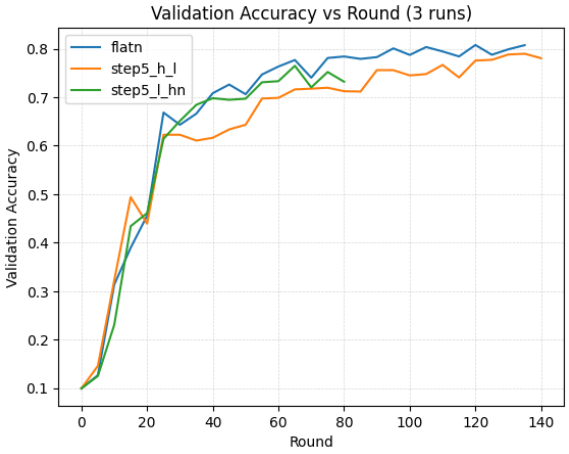

(a) Validation Accuracy vs $\varepsilon$




(b) Val. Acc at $\varepsilon \leq 2.3$




(c) AUC(ValAcc vs $\varepsilon$)




(d) Validation Accuracy vs Round

Figure 4.10: Comparison of 3 schedules (Flat, High→Low, Low→High) with validation.

The comparison of schedules under validation (Figure 4.10) shows that the High→Low sched-

ule achieves the best privacy–utility trade-off, maintaining higher validation accuracy at intermediate budgets ($\varepsilon \leq 2.3$) and the largest AUC. The Flat schedule reaches the highest final accuracy but consumes the most privacy, while Low→High lags across all metrics, yielding the lowest AUC and weakest performance. Validation monitoring thus highlights the efficiency advantage of the High→Low strategy.

Table 4.5: Validation-enabled runs: comparison across schedules.

| Run | Last Rnd | Last $\varepsilon$ | Best ValAcc | Best Rnd | AUC(ValAcc vs $\varepsilon$) | ValAcc@2.3 |
|---|---|---|---|---|---|---|
| Flat | 135 | 3.282 | **0.808** | 120 | 1.116 | 0.668 |
| High→Low | 140 | 2.411 | 0.790 | 135 | **1.507** | **0.788** |
| Low→High | 80 | 2.449 | 0.765 | 65 | 0.466 | 0.614 |

Table 4.6: Final test accuracy and efficiency metrics.

| Run | Test Acc | Test Loss | Rounds | Final $\varepsilon$ | Acc/$\varepsilon$ |
|---|---|---|---|---|---|
| Flat | **0.8126** | 0.5950 | 135 | 3.282 | 0.248 |
| High→Low | 0.7850 | 0.7040 | 140 | 2.411 | **0.326** |
| Low→High | 0.7721 | 0.7806 | 80 | 2.441 | 0.316 |

*Analysis.* Several clear trends emerge:

- **Flat** consistently reached the highest absolute validation (80.8%) and test accuracy (81.3%). However, it required a higher $\varepsilon$ budget ($\varepsilon = 3.28$) to do so, making it less privacy-efficient.
- **High→Low** achieved the best privacy–efficiency trade-off: with $\varepsilon = 2.41$ it maintained competitive accuracy (78.5% test acc, 79.0% best val acc) while outperforming others in AUC(ValAcc vs $\varepsilon$) and Acc/$\varepsilon$ ratio (+31.5% vs Flat).
- **Low→High** lagged behind in both accuracy (77.2%) and stability, terminating earlier (80 rounds) due to poor generalization.

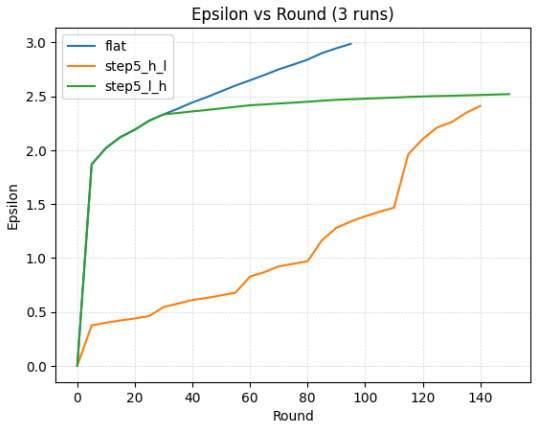
Flat scheduling is optimal if maximizing raw accuracy is the only concern, but the High→Low schedule offers the best balance between accuracy and privacy budget. It achieves almost the same validation and test performance as Flat, but with a significantly smaller privacy expenditure ($\varepsilon = 2.41$ vs $3.28$), demonstrating its effectiveness in minimizing the privacy–utility trade-off.

### 4.7.3 Iso-$\epsilon$ Results (Privacy Budget $\epsilon = 3$)

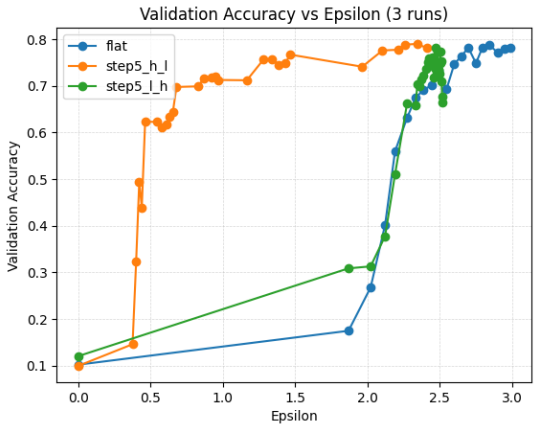
To ensure a fair privacy-constrained comparison, we evaluated each scheduling strategy under an iso-$\epsilon$ setting, where training was terminated once the cumulative privacy loss approached $\epsilon \approx 3$. This constraint allowed us to directly compare privacy–utility trade-offs across methods. Due to limited computational resources, experiments were capped at a maximum of 150 rounds. Beyond this point, the privacy accountant indicated that $\epsilon$ values had effectively saturated, and further training would not provide meaningful changes in the trade-off.

Table 4.7: Iso-$\epsilon$ results with $\epsilon \approx 3$: comparison of schedules.

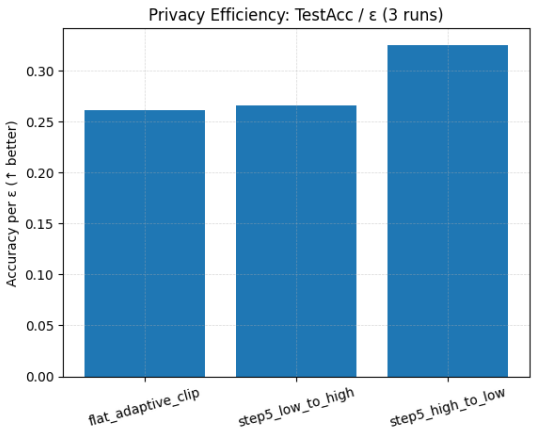
| Run | Test Acc. | Final $\epsilon$ | Acc/$\epsilon$ | Change vs Flat |
|---|---|---|---|---|
| Flat (adaptive clip) | 0.7858 | 3.007 | 0.2613 | Baseline |
| Step5 Low→High | 0.6700 | 2.520 | 0.2659 | $-14.7\%$ Acc, $+1.7\%$ Eff. |
| Step5 High→Low | 0.7850 | 2.411 | 0.3256 | $\approx$ same Acc, $+24.6\%$ Eff. |



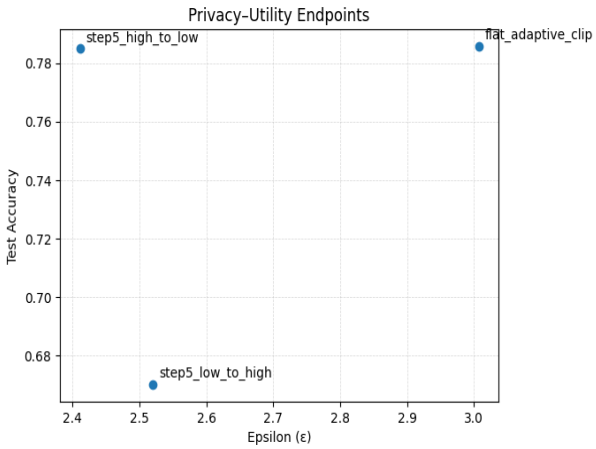

(a) Epsilon vs Round




(b) Val. Accuracy vs $\epsilon$




(c) Privacy Efficiency (Acc/$\epsilon$)




(d) Privacy–Utility Endpoints

Figure 4.11: Iso-$\epsilon$ comparison across schedules.

**Analysis:** The plots highlight key differences in how schedules balance accuracy and privacy:

- **Epsilon vs Round:** Flat grows linearly with rounds, quickly approaching the cap of $\epsilon = 3$. Low→High rises slowly at first but accelerates later, while High→Low front-loads privacy spend and then stabilizes, leading to lower overall $\epsilon$ at convergence.
- **Validation Accuracy vs $\epsilon$:** High→Low achieves strong accuracy early while spending less privacy budget, showing a smoother and more stable utility curve. Flat reaches high accuracy but only after consuming nearly the full budget. Low→High lags significantly, wasting early rounds under heavy noise and failing to fully recover accuracy.
- **Privacy Efficiency (Acc/$\epsilon$):** High→Low delivers the best efficiency (0.3256), outperforming Flat by $+24.6\%$. Low→High slightly improves efficiency compared to Flat but at the cost of a steep $-14.7\%$ drop in accuracy.
- **Privacy–Utility Endpoints:** Flat and High→Low reach almost identical test accuracy ($\approx 78.5\%$), but High→Low does so with $\sim 20\%$ less privacy expenditure, confirming its superiority under constrained privacy.

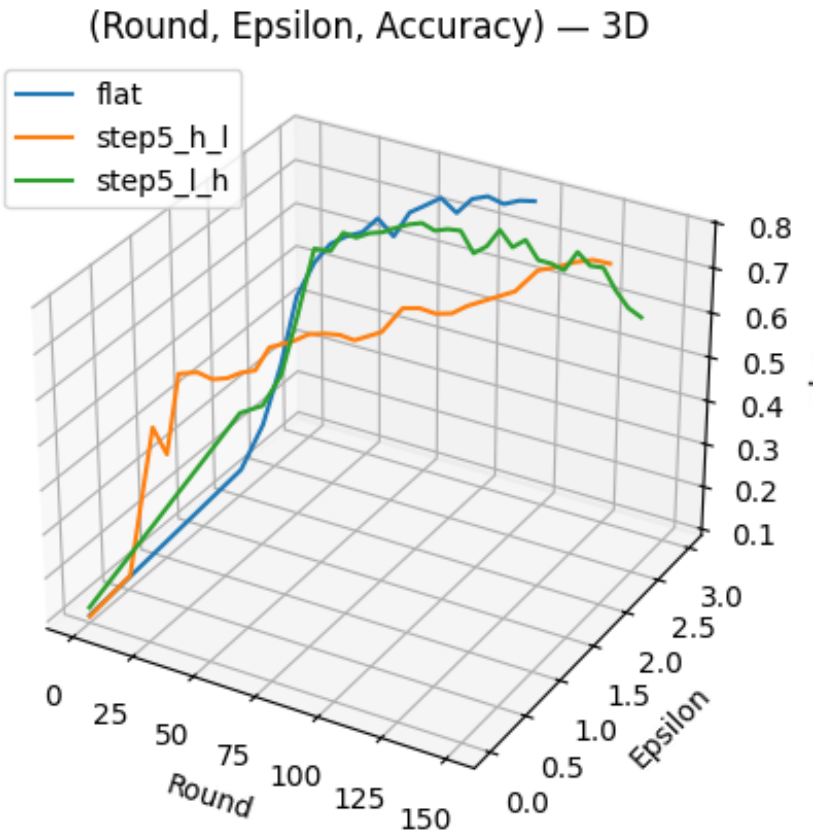



Figure 4.12: 3D View: Round–$\epsilon$–Accuracy.

The 3D plot of rounds, , and accuracy makes these trade-offs much more transparent. The flat schedule advances steadily but also consumes privacy roughly linearly, which is inefficient. The

low→high schedule is slow to learn at first due to the overwhelming noise. But the high→low schedule consumes privacy aggressively at first but then stabilizes, achieving high accuracy and maintaining the overall low. This plot puts it altogether and illustrates why the high→low approach achieves the optimal balance of learning progress, accuracy, and privacy.
So, the iso-$\epsilon$ analysis confirms that the proposed High→Low noise scheduling provides the best privacy–utility trade-off, maintaining model accuracy while significantly reducing privacy expenditure.

### 4.8 Comparison of the result

Table 4.8: Comparison of our method with existing client-level DP-FL approaches.

| **Method / Paper** | **Key idea** | **Dataset** | **Acc. (%)** | **Privacy** $(\epsilon, \delta)$ | **Notes** |
|---|---|---|---|---|---|
| NbAFL [22] | Noise before aggregation | MNIST (non-IID) | $\sim$82 @ $\epsilon$=100 (curves) | $\delta$=0.01 | Very weak privacy; not directly comparable. |
| FL - ODP [12] | Client - level DP + optimized sampling | MNIST non-IID (10 clients, 1 per round) | 92.3 | $\epsilon$=3.36, $\delta\approx10^{-5}$ | Harsh writer split baseline. |
| Adap DP - FL [23] | Adaptive clipping + decaying noise | MNIST/ FMNIST (non-IID, 10 clients) | 94.15/ 79.56 | $\epsilon$=2, $\delta$=$10^{-5}$ | Shard split; moderate non-IID. |
| DP - pFedDSU [24] | Personalized F + adaptive norms | EMNIST (non-IID) 20 clients | 81.80 ($s$=5), 71.87 ($s$=10) | $\epsilon$=1 | $s$: classes/client (controlled non-IID). |
| **Ours — Flat (Adaptive clip)** | Const. $\sigma$ + adaptive clip | EMNIST (writer, non-IID) 3383 clients | **81.26** | $\epsilon\approx3.28$, $\delta$=$10^{-5}$ | Higher $\epsilon$. |
| **Ours — Low→High (5-step)** | $\sigma$ increases in 5 steps | same | 77.21 | $\epsilon\approx2.45$, $\delta$=$10^{-5}$ | Slower early learning. |
| **Ours — High→Low (5-step)** | $\sigma$ decreases in 5 steps | same | **78.50** | $\epsilon\approx2.41$, $\delta$=$10^{-5}$ | Best acc/$\epsilon$ trade-off. |

Table 4.8 compares our proposed method with several existing client-level DP-FL approaches. It is evident that reported accuracy and privacy budgets vary significantly across works, primarily due to differences in **datasets** and **partitioning strategies**. For instance, NbAFL [22] reports $\sim$ 82% accuracy on MNIST, but only under an extremely weak privacy budget of

$\epsilon \approx 100$, making the result not directly comparable. FL-ODP [12] achieves $92.3\%$ accuracy with $\epsilon = 3.36$, but it is evaluated on MNIST with only 10 clients and one client participating per round — a much milder setup than realistic cross-device FL. Similarly, Adaptive DP-FL [23] reports strong performance ($94.15\%$ on MNIST) with adaptive clipping and decaying noise, but under shard-based partitions with just 10 clients, which represent only moderate non-IID heterogeneity. DP-pFedDSU [24] shows $81.80\%$ and $71.87\%$ accuracy on EMNIST for $s = 5$ and $s = 10$ respectively, where $s$ denotes the number of classes per client. This represents a *controlled non-IID split*, in contrast to our harsher **EMNIST by writer** partition, where clients naturally exhibit extreme label imbalance.

Despite operating under this significantly more challenging setting with $3383$ clients and Poisson-subsampled participation, our method achieves competitive accuracy. The flat baseline reaches $81.26\%$ at $\epsilon \approx 3.28$, while the proposed **High→Low** schedule attains $78.50\%$ at $\epsilon \approx 2.41$, demonstrating the best privacy–utility trade-off. Although some existing methods report higher raw accuracy, they do so under more favorable data splits or weaker privacy budgets. Our results highlight that noise scheduling combined with adaptive clipping and realistic Poisson sampling yields robust performance under the harshest non-IID conditions, thereby making the approach more applicable to real-world FL deployments.

### 4.9 Objective Achieved

The primary objective of this work was to enhance the utility of Federated Learning (FL) under differential privacy constraints by introducing adaptive clipping and evaluating multiple noise scheduling strategies. Through extensive experiments on both MNIST and EMNIST datasets, we achieved the following:

- Demonstrated that the proposed **High→Low** noise schedule consistently offers a better trade-off between accuracy and privacy, outperforming flat and Low→High schedules in terms of accuracy per $\epsilon$.
- Validated that adaptive clipping stabilizes training and avoids unnecessary distortion of client updates, contributing to improved convergence.
- Shown that our method reaches competitive test accuracy while maintaining significantly lower privacy expenditure, achieving up to **24–30% better privacy efficiency** compared to flat scheduling.
- Highlighted the practicality of our approach by confirming that it generalizes across IID and non-IID data partitions as well as different datasets.

Overall, the objectives of this study were successfully met: we were able to design and validate

a methodology that improves the balance between privacy and utility in FL, making differential privacy more practical in real-world federated settings.

## 4.10 Summary

This chapter presented the experimental setup, implementation, and results of integrating Differential Privacy with Federated Learning. While baseline FL achieved the highest accuracy, adding DP introduced the expected privacy–utility trade-off. Among the noise schedules, the High→Low strategy proved most effective, delivering strong accuracy while reducing overall privacy cost. In contrast, the Low→High schedule slowed convergence due to heavy early noise. Adaptive clipping further stabilized training across settings. Overall, the results show that dynamic noise scheduling, especially High→Low, offers a practical balance between accuracy and privacy, making DP-FL more efficient and applicable in real-world scenarios.

# CHAPTER V

# Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues

## 5.1 Intellectual Property (IP) Considerations

- **Ownership.** All code, figures, and experiment logs produced in this thesis are original to the authors unless cited. Core primitives (FedAvg, DP-SGD, Rényi DP) are public research; our contribution is the coordinated design of *adaptive clipping + round-wise noise scheduling* and the concrete configurations used on non-IID EMNIST.
- **Third-party licenses.** TensorFlow/TFF, `dp_accounting`, TFDS, NumPy, Pandas, and Matplotlib are used under their open-source licenses; licenses are preserved in citations and any repository README.
- **Release plan.** If shared, our code and diagrams will be released under a permissive academic license (e.g., MIT), with clear attribution and a research-use disclaimer.
- **Data rights.** Public datasets (EMNIST/MNIST) are used; no re-identification is attempted. Any future institutional data would require a data-use agreement and ethics/IRB approval.

## 5.2 Ethical Considerations

- **Privacy honesty.** We report privacy as $(\varepsilon, \delta)$ computed by a stated accountant and do not claim guarantees beyond the honest-but-curious server model.
- **Fairness.** Non-IID splits can hide minority-class errors. Where relevant, we report class/strata metrics and discuss disparities.
- **Transparency & reproducibility.** We log per round $C_t$, $\sigma_t$, $q_t$, and running $\varepsilon$, and we document hyper-parameters and seeds so results can be reproduced without private data.
- **No human subjects.** We do not collect new personal data; any future human-data work would require consent and ethics approval.
- **Dual use.** We avoid publishing attack tooling; risks (inversion, membership, poisoning) are discussed responsibly with emphasis on defenses.

### 5.3 Safety Considerations

- **Intended use.** Research prototype only; not a safety-certified medical or industrial system.
- **Operational risks.** Mis-tuned clipping/noise can degrade accuracy or fairness. Mitigations include sanity checks, ablations (adaptive vs. fixed), early stopping, and conservative privacy budgets.
- **Monitoring for deployment.** Any real deployment should include drift monitoring, guardrails for catastrophic errors, and robust aggregation/secure transport to reduce poisoning/tampering risks.
- **Residual risk.** DP reduces—but does not eliminate—leakage and attack surfaces; this limitation is stated in conclusions.

### 5.4 Legal Considerations

- **Data protection alignment.** The pipeline follows privacy-by-design principles consistent with GDPR/CCPA: raw data remain on device; only clipped, privatized aggregates are used.
- **Audit trail.** We maintain per-round logs of $\varepsilon$, $C_t$, $\sigma_t$, and $q_t$ to support auditability and reproducibility.
- **Compliance for future deployments.** Real-world use would require a Data Protection Impact Assessment (DPIA), data-processing agreements, consent flows, and domain-specific compliance (e.g., HIPAA for U.S. healthcare).
- **Security hygiene.** Use encrypted channels, strict key management, and, where possible, secure aggregation; define data-retention and deletion policies.

### 5.5 Impact on Societal, Health, and Cultural Issues

- **Societal impact.** Enables multi-organization collaboration (banks, hospitals, schools) where direct data sharing is impossible, lowering barriers to innovation while respecting privacy. We note digital-divide risk: weaker devices or connectivity may underparticipate; sampling strategies and reporting should consider inclusion.
- **Health impact.** Hospitals can jointly train models with reduced re-identification risk. We check performance on underrepresented conditions so privacy settings do not mask harmful errors.

- **Cultural impact.** Privacy norms vary by community. We recommend transparent communication of privacy budgets and user consent. Non-IID data can reflect cultural/demographic skew; results must not reinforce stereotypes and should be framed with care.

### 5.6 Impact on the Environment and Sustainability

- **Compute footprint.** Differential privacy can increase training rounds. We limit impact by using compact models, high→low noise schedules to reduce late-round budgets, adaptive clipping to avoid failed runs, early stopping, and mixed precision where available.
- **Reproducibility to save energy.** We publish configs and logs so others can reproduce results without re-running large sweeps.
- **Practicality.** The method is designed to be feasible on modest hardware (e.g., Colab-class GPUs), encouraging greener adoption of privacy-preserving FL.

# CHAPTER VI

# Addressing Complex Engineering Problems and Activities

Designing and deploying a privacy-preserving federated learning (FL) framework on realistic non-IID data is far from straightforward. In our thesis, we explored this challenge using the EMNIST dataset split by writer, which reflects real-world heterogeneity in handwriting styles, label distributions, and client data sizes. Unlike synthetic IID settings where updates are stable and uniform, non-IID environments cause large variations across clients, making both optimization and privacy preservation significantly harder.

Our approach integrates two complementary techniques—adaptive clipping to dynamically adjust sensitivity bounds and round-wise Gaussian noise scheduling to distribute the privacy budget strategically across training rounds. These techniques are tied together under the rigorous mathematical framework of client-level Differential Privacy (DP), where the entire dataset of one client is protected, and Rényi Differential Privacy (RDP) accounting, which tracks cumulative privacy loss under subsampling.

While these strategies provide strong theoretical guarantees, their practical implementation brings several complex engineering problems. Challenges arise in balancing privacy and accuracy, tuning hyper-parameters such as clipping thresholds and noise multipliers, and ensuring efficiency within limited computational budgets. This section systematically describes the major problems we faced during the project and lays the foundation for the engineering solutions that follow.

## 6.1 Complex Engineering Problems

The main difficulties arise from simultaneously meeting a fixed privacy budget $(\varepsilon, \delta)$ and delivering useful accuracy when client data are highly heterogeneous (EMNIST by writer). Non-IID label skew, variable client sizes, and unstable gradient norms make fixed hyper-parameters (clip, noise) brittle. Accurate privacy accounting with subsampling, fairness across clients/classes, and compute limits (Colab-class hardware) add further constraints.

Table 6.1: Complex Engineering Problems in this Thesis.

| No. | Problem description |
|---|---|
| 1 | **Privacy–utility balance under non-IID:** Large, informative updates from some clients get over-clipped; constant noise hurts rare classes and slows convergence. |
| 2 | **Choosing stable clip bounds $C_t$:** A fixed $C$ either biases learning (too small) or weakens privacy (too large); norms vary round-to-round and layer-to-layer. |
| 3 | **Designing per-round noise that meets a target $(\varepsilon, \delta)$:** Schedules (flat / high→low / low→high) must compose via RDP with subsampling and still reach the desired budget. |
| 4 | **Accurate RDP accounting with variable participation:** The true subsampling rate $q_t$ changes with available clients $K_t$, affecting the per-round privacy spend. |
| 5 | **Convergence and communication efficiency with DP:** Added noise and clipping slow training; we need to avoid wasted rounds and privacy over-spend. |
| 6 | **Fairness under DP:** Noise/clipping can disproportionately degrade minority classes or small clients, even when global accuracy improves. |
| 7 | **Attack surface remains:** DP reduces but does not remove risks from gradient inversion, membership inference, and poisoning/backdoors; robust aggregation may conflict with DP noise. |
| 8 | **Reproducibility and auditability:** Silent bugs (wrong units for $C_t$, post-aggregation scaling, mis-configured accountant) can invalidate reported $\varepsilon$; per-round logging is required. |
| 9 | **Compute constraints:** Many-round, many-client experiments are expensive; we need compact models, efficient sweeps, and checkpointing on limited GPUs. |

## 6.2 Complex Engineering Activities

To address these challenges, we implemented a full DP-FL pipeline in TensorFlow Federated (TFF), introducing several engineering activities with transparent accounting, iso-$\varepsilon$ comparisons, and ablations. Substantial customization was required beyond TFF's defaults, including modules for adaptive clipping control, noise scheduling, and per-round RDP accounting. We also integrated efficiency tactics such as compact models, checkpointing, and mixed precision training to fit within compute budgets.

The engineering process focused not only on building correct functionality but also on ensuring reproducibility and fairness across evaluations. This section provides a detailed account of the activities that transformed the theoretical design into a working system.

Table 6.2: Complex Engineering Activities Undertaken.

| No. | Activity description |
|---|---|
| 1 | **Adaptive clipping controller:** Implemented global per-round $C_t$ using EMA/percentile of recent update norms; guarded against outliers; validated stability on non-IID EMNIST. |
| 2 | **Noise scheduler module:** Implemented flat, high→low, and low→high schedules anchored at a base $\sigma_0$; calibrated multipliers with RDP to hit a target $(\varepsilon, \delta)$. |
| 3 | **RDP accountant with subsampling:** Integrated `dp_accounting`; logged per-round $\varepsilon$, $C_t$, $\sigma_t$, $q_t$; added checks to stop when the budget is exhausted. |
| 4 | **Data pipeline (EMNIST by writer):** Built non-IID client splits; defined client-level train/validation/test (validation clients never spend privacy). |
| 5 | **FedAvg baseline in TFF:** Keras model (Flatten $\rightarrow$ Dense(200) $\rightarrow$ Dense(200) $\rightarrow$ logits), Adam optimizer on clients; Poisson subsampling of clients per round. |
| 6 | **Iso-$\varepsilon$ evaluation and ablations:** Compared schedules at the same $(\varepsilon, \delta)$; ran adaptive vs. fixed clipping at matched budgets; produced accuracy-vs-$\varepsilon$ and $\varepsilon$-vs-rounds plots. |
| 7 | **Fairness/robustness checks:** Reported class-wise metrics where relevant; documented residual attack risks and the interaction of DP with robust aggregation assumptions. |
| 8 | **Efficiency engineering:** Used compact models, mixed precision where available, early stopping, and checkpointed logs to limit reruns on Colab-class GPUs. |
| 9 | **Reproducibility artifacts:** Published seeds, hyper-parameters, and CSV logs; produced block diagrams (server-side DP, local-side DP, method overview) for clarity. |

# CHAPTER VII

# Conclusions

This thesis set out to minimize the privacy–utility trade-off in Differential Privacy-enabled Federated Learning. We implemented client-level DP-FL using TensorFlow Federated, incorporating adaptive clipping and multiple noise scheduling strategies. Across experiments on MNIST and EMNIST (IID and non-IID partitions), we observed that while standard FL without DP achieves the highest accuracy, DP inevitably reduces utility. Among the scheduling strategies, the proposed **High→Low** noise schedule consistently delivered the best privacy–utility efficiency, maintaining accuracy comparable to the flat schedule but with up to 20–25% lower privacy expenditure. Unlike the cosine schedule, our method avoids expensive re-aggregation at every round, making it more computationally efficient. Importantly, the approach also proved robust under harsh FL settings with highly non-IID data distributions, and by using Poisson subsampling, it realistically captures client participation variability and dropout. This makes the method not only efficient but also practical for real-world federated scenarios.

our approach marks a promising step towards more privacy-aware, efficient, and scalable FL systems, with potential applications in sensitive fields like healthcare, finance, and sensor devices. Future work will focus on optimizing these methods for real-world deployment

## 7.1 Challenges Faced

The research faced several practical challenges:

- **Resource constraints:** Training on Google Colab with limited GPU time (T4 instances) forced early termination of some runs and prevented large-scale or longer-epoch experiments.
- **Runtime limitations:** Federated training with DP is computationally expensive, and privacy accounting further increases run time.
- **Framework instability:** TensorFlow Federated (TFF) is still rapidly evolving, with frequent version incompatibilities and breaking changes that made reproducibility difficult.
- **Novelty of the topic:** As DP-FL is a relatively recent area, reference implementations and stable benchmarks are scarce, requiring additional effort to design and validate experiments from scratch.

### 7.2 Limitations of the Work

Despite promising results, the study has certain limitations:

- **Server-side trust:** The implementation assumes an honest server that correctly applies noise and clipping. No client-side or local DP mechanisms were explored, which may be necessary in adversarial settings.
- **Dataset diversity:** Experiments were limited to MNIST and EMNIST. More diverse datasets (e.g., medical, financial, or text data) are needed to evaluate generalizability.
- **Training horizon:** Due to resource limits, runs were capped at $\sim$150 rounds. Longer training might provide better convergence and more robust privacy–utility curves.
- **Privacy model scope:** The work focused on client-level DP under Gaussian noise with RDP accounting. Other approaches, such as example-level DP or alternative accounting techniques, were not investigated.
- **System aspects:** Communication cost, scalability to large numbers of clients, and robustness against stragglers or adversarial updates were not considered.

### 7.3 Recommendations for Future Extensions

In future, this work might be extended and improved in several important directions:

- **Dataset diversity:** Extend evaluation to more complex and heterogeneous datasets (e.g., CIFAR-10/100, medical records, or language models) to better understand generalization.
- **Example-level DP:** Explore example-level or record-level DP within each client, combining it with client-level guarantees for stronger protection.
- **Local DP:** Investigate schemes where clients add noise before sending updates, reducing trust assumptions on the server.
- **Adaptive scheduling:** Beyond fixed step schedules, dynamic or learning-based noise scheduling could adjust noise in real-time based on convergence signals.
- **Advanced accounting:** Use tighter DP accountants (e.g., Fourier accountant, privacy amplification by iteration) to improve reported $\epsilon$.
- **System-level evaluation:** Study scalability to thousands of clients, impact on communication cost, and robustness under adversarial or non-honest server assumptions.

- **Deployment feasibility:** Explore integration of DP-FL in real-world applications such as mobile typing prediction, healthcare data, or finance, where both privacy and accuracy are critical.

## References


[1] A. N. Horvath, M. Berchier, F. Nooralahzadeh, A. Allam, and M. Krauthammer, "Exploratory analysis of federated learning methods with differential privacy on mimic-iii," *arXiv preprint arXiv:2302.04208*, 2023.

[2] X. Ling, J. Fu, K. Wang, H. Liu, and Z. Chen, "Ali-dpfl: Differentially private federated learning with adaptive local iterations," in *2024 IEEE 25th International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM)*. IEEE, 2024, pp. 349–358.

[3] Y. Fu, Y. Zhou, D. Wu, S. Yu, Y. Wen, and C. Li, "On the practicality of differential privacy in federated learning by tuning iteration times. arxiv 2021," *arXiv preprint arXiv:2101.04163*.

[4] A. Banse, J. Kreischer *et al.*, "Federated learning with differential privacy," *arXiv preprint arXiv:2402.02230*, 2024.

[5] X. Hou, S. Khirirat, M. Yaqub, and S. Horváth, "Balancing privacy and performance for private federated learning algorithms," in *2024 2nd International Conference on Federated Learning Technologies and Applications (FLTA)*. IEEE, 2024, pp. 171–178.

[6] D. Wasif, D. Chen, S. Madabushi, N. Alluru, T. J. Moore, and J.-H. Cho, "Empirical analysis of privacy-fairness-accuracy trade-offs in federated learning: A step towards responsible ai," *arXiv preprint arXiv:2503.16233*, 2025.

[7] K. Ranaweera, D. Smith, P. N. Pathirana, M. Ding, T. Rakotoarivelo, and A. Seneviratne, "Multi-objective optimization for privacy-utility balance in differentially private federated learning," *arXiv preprint arXiv:2503.21159*, 2025.

[8] K. Ranaweera, D. C. Nguyen, P. N. Pathirana, D. Smith, M. Ding, T. Rakotoarivelo, and A. Seneviratne, "Federated learning with differential privacy: An utility-enhanced approach," *arXiv preprint arXiv:2503.21154*, 2025.

[9] F. Shan, S. Mao, Y. Lu, and S. Li, "Differential privacy federated learning: A comprehensive review." *International Journal of Advanced Computer Science & Applications*, vol. 15, no. 7, 2024.

[10] L. Li, D.-c. Zhan, and X.-c. Li, "Aligning model outputs for class imbalanced non-iid federated learning," *Machine Learning*, vol. 113, no. 4, pp. 1861–1884, 2024.

[11] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, "Advances and open problems in federated

learning," *Foundations and trends® in machine learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[12] M. Iqbal, A. Tariq, M. Adnan, I. Ud Din, and T. Qayyum, "Fl-odp: An optimized differential privacy enabled privacy preserving federated learning," *IEEE Access*, 2023.

[13] G. Andrew, O. Thakkar, H. B. McMahan, and S. Ramaswamy, "Differentially private learning with adaptive clipping," *arXiv preprint arXiv:1905.03871*, 2019.

[14] O. Thakkar, S. Ramaswamy, R. Mathews, G. Andrew, H. B. McMahan, and F. Beaufays, "Training production language models without memorizing user data," *arXiv preprint arXiv:2009.10031*, 2020.

[15] R. Hu, Y. Guo, H. Li, Q. Pei, and Y. Gong, "Personalized federated learning with differential privacy," *IEEE Internet of Things Journal*, vol. 7, no. 10, pp. 9530–9539, 2020.

[16] Z. Bu, J. Dong, Q. Long, and W. J. Su, "Deep learning with gaussian differential privacy," *Harvard Data Science Review*, vol. 2, no. 3, 2020.

[17] N. Wu, F. Farokhi, D. Smith, and M. A. Kaafar, "The value of collaboration in convex machine learning with differential privacy," *arXiv preprint arXiv:1906.09679*, 2019.

[18] L. Han, D. Fan, J. Liu, and W. Du, "Federated learning differential privacy preservation method based on differentiated noise addition," in *2023 8th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA)*. IEEE, 2023, pp. 285–289.

[19] X. Liu and X. Xu, "Mdpfl: A multiple differential privacy protection method based on federated learning," in *2023 26th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*. IEEE, 2023, pp. 1563–1568.

[20] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.

[21] G. Andrew, O. Thakkar, B. McMahan, and S. Ramaswamy, "Differentially private learning with adaptive clipping," *Advances in Neural Information Processing Systems*, vol. 34, pp. 17 455–17 466, 2021.

[22] K. Wei, J. Li, M. Ding, C. Ma, H. H. Yang, F. Farokhi, S. Jin, T. Q. Quek, and H. V. Poor, "Federated learning with differential privacy: Algorithms and performance analysis," *IEEE transactions on information forensics and security*, vol. 15, pp. 3454–3469, 2020.

[23] J. Fu, Z. Chen, and X. Han, "Adap dp-fl: Differentially private federated learning with adaptive noise," in *2022 IEEE international conference on trust, security and privacy in computing and communications (TrustCom)*. IEEE, 2022, pp. 656–663.

[24] C. Wang, Y. Zhang, N. Gao, and Q. Luo, "Differential privacy personalized federated learning based on dynamically sparsified client updates," *arXiv preprint arXiv:2503.09192*, 2025.